

Table of Contents

1. Introduction	1
1.1 Project Overview	1
1.2 Current Scenario	1
1.3 Problem Statement.....	2
1.4 Aim and Objectives.....	2
1.5 Scope of the Project	3
2. Background and Requirement Analysis	4
2.1 Introduction to Smart Security Systems.....	4
2.2 Traditional Door Locking Systems	4
2.3 IoT-Based Security Systems	5
2.4 Multi-Factor Authentication Systems	6
2.5 Existing System Limitations.....	7
2.6 Requirement Analysis	8
2.6.1 Functional Requirements	8
2.6.2 Non-Functional Requirements	9
3.1 System Overview.....	10
3.2 System Design and Architecture Diagram	11
3.3 Flowchart of the System	12
3.4 Hardware Components.....	13
3.4.1 Raspberry Pi 5	13
3.4.2 USB Webcam	13
3.4.3 RC-522 RFID Module	14
3.4.4 16×2 I2C LCD Display	14
3.4.5 R307S Fingerprint Sensor.....	15

3.4.6 Piezoelectric Buzzer	15
3.4.7 12V Electromagnetic Door Lock.....	15
3.4.8 5V Relay Module 2 Channel Relay Module	16
3.5 Software and Tools	16
3.5.1 Thonny IDE	16
3.5.2 Python 3.....	17
3.5.3 OpenCV Library	18
3.5.4 NumPy	18
3.5.5 RPi.GPIO	18
3.5.6 MFRC522 Library.....	19
3.6 Communication and Cloud Integration	19
3.6.1 MQTT Protocol.....	19
3.6.2 HiveMQ Integration	20
4. Development Process	22
4.1 Development Methodology	22
4.2 Stage 1 – Hardware Setup and Raspberry Pi Configuration	22
4.3 Stage 2 – RFID Authentication Module Development.....	23
4.4 Stage 3 – Fingerprint Authentication Module Development.....	24
4.5 Stage 4 – Face Recognition Module Development.....	24
4.6 Stage 5 – LCD, LEDs and Buzzer Integration	25
4.7 Stage 6 – Electromagnetic Door Lock and Relay Control.....	25
4.8 Stage 7 – MQTT Communication Setup	26
4.9 Stage 8 – HiveMQ Cloud Integration	26
4.10 Final System Integration	27
5. Results and Findings.....	27
RASHIK SHRESTHA	ii

5.1 Testing Overview	27
5.2 Unit Testing.....	29
5.2.1 RFID Module Testing.....	29
5.2.2 Fingerprint Sensor Testing	32
5.2.3 Face Recognition Testing.....	38
5.2.4 Relay Module Testing.....	38
5.2.5 LCD and Buzzer Testing	42
5.3 System Testing	45
5.3.1 Valid Authentication Test	45
5.3.2 Invalid RFID Test.....	47
5.3.3 Fingerprint Mismatch Test.....	49
5.3.4 Face Detection Failure Test	50
5.3.5 MQTT Communication Test	52
5.4 Failed Test Cases and Challenges	55
5.4.1 Face Recognition Fail Case	55
5.5 Results Analysis and Discussion	58
5.5.1 Multi-Threaded Asynchronous Architecture and Edge Performance.....	58
5.5.2 Fault Isolation and Self-Healing Capability	59
5.5.3 Cloud Telemetry and Security Hardening Synergies.....	61
6. Further Work	61
6.1 Mobile Application Integration.....	61
6.2 Remote Monitoring System	62
6.3 AI-Based Threat Detection.....	62
6.4 Database Integration	62
6.5 Backup Power System	63
RASHIK SHRESTHA	iii

6.6 Mobile Notification System	63
7. Conclusion	64
8. References.....	64

Table of Figures

Figure 1 : System Architecture Diagram for the smart door-lock system	13
Figure 2 : Flowchart for the smart door lock system	14
Figure 3 : Raspberry Pi 5 8GB	15
Figure 4 : A 4MP USB Webcam	15
Figure 5 : RC-522 RFID Module	16
Figure 6 : 16 x 2 I2C LCD display	17
Figure 7 : R307S Fingerprint module	17
Figure 8 : Piezoelectric buzzer	17
Figure 10 : 12V Electromagnetic Door lock	18
Figure 11 : 5V relay module	18
Figure 12 : Thonny IDE	19
Figure 13 : Python3 logo	20
Figure 14 : OpenCV Logo	20
Figure 15 : NumPy Logo	20
Figure 16 : RPi.GPIO python library	21
Figure 17 : MFRC522 python library.	21
Figure 18 : MQTT Publish / Subscribe Architecture	22
Figure 19 : MQTT connection details.	23
Figure 20 : MQTT user login	23
Figure 21 : Accessible MQTT WebUI.	24
Figure 22 : RFID module lights up when connected.	33

Figure 23 : Running program to read RFID card.	34
Figure 24 : Physically scanning RFID card.	34
Figure 25 : Card's UID appears and is successfully read.	35
Figure 26 : Fingerprint module turns on when powered.	37
Figure 27 : Program ready to read and test fingerprint.	38
Figure 28 : Physically scanning fingerprint.	39
Figure 29 : Fingerprint is detected but is not registered.	40
Figure 30 : Door lock bolt is extended when unpowered.	43
Figure 31 : Program is run and the state is changed from OFF to ON	43
Figure 32 : Relay module lights up and the bolt is retracted (success).	44
Figure 33 : I2C LCD module powers up when connected.	46
Figure 34 : Program scans for RFID to display its UID.	46
Figure 35 : I2C module displays the UID for the RFID (success).	47
Figure 36 : Buzzer beeps when a RFID card is scanned.	47
Figure 37 : Successfully detects face.	49
Figure 38 : The program unlocked the door.	49
Figure 39 : Logs face unlock event to Hive MQ.	49
Figure 40 : Door unlocks with registered fingerprint.	50
Figure 41 : Finger unlock event is logged in Hive MQ.	50

Figure 42 : Authorized RFID is detected then the door unlocks.	50
Figure 43 : RFID unlock event is logged in Hive MQ.	50
Figure 44 : When valid and registered RFID card is scanned.	52
Figure 45 : When unregistered RFID card is scanned.	52
Figure 46 : Unregistered fingerprint is not recognized.	53
Figure 47 : Unauthorized login event is logged in Hive MQ.	53
Figure 48 : Unregisterd face is not detected, the door remains locked.	55
Figure 49 : Program conncts to MQTT and starts.5.3.6 AWS Cloud Logging Test.	57
Figure 50 : The Hive MQ Cluster recieves and logs all event logs.	59
Figure 51 : Unregistered user was allowed access.....	62

List of Tables

Table 1 : RFID Module Testing	32
Table 2 : Fingerprint Sensor Testing	36
Table 3 : Face Recognition Testing	41
Table 4 : Relay Module Testing	42
Table 5 : LCD and Buzzer Testing	45
Table 6 : Valid Authentication Test	48
Table 7 : Invalid RFID Test	51
Table 8 : Fingerprint Mismatch Test	53
Table 9 : Face Detection Failure Test	54
Table 10 : MQTT Communication Test	56
Table 11 : AWS Cloud Logging Test	58

1. Introduction

In recent years, technology has rapidly evolved and has thus made a considerable impact to revolutionize conventional security systems to being more intelligent and effective. Traditional lock and key methods have slowly been replaced by smart systems that offers greater security, comfort and versatility to the system. In the form of smart access control, these intelligent systems are incorporated to various locations such as home, office and secured area that restrict illegal access.

1.1 Project Overview

Technology today has enabled high-security systems to eliminate the need for traditional lock-and-key mechanisms

1.2 Current Scenario

In most cases, the security systems used for residential and small-scale purposes still depend on traditional locks or use one-factor authentication systems such as PIN codes or RFID cards. However, despite their simplicity, there are several drawbacks to these systems. Keys can be lost, stolen, or shared. RFID cards or passwords can also be shared or compromised.

Although there are advanced security systems that can authenticate users based on their biometric information, these technologies are expensive and not easily accessible for small-scale or personal use. Second, there is only one form of authentication, which makes it impossible to use the system entirely if the authentication method is compromised or bypassed.

With the rise of smarter and more secure environments, there is a demand for cheap, multi-factor authentication systems that can ensure better protection while not disrupting usability.

1.3 Problem Statement

Traditional door locking systems and existing single-factor authentication mechanisms are inadequate to assure modern security needs. The traditional door locking system as well as the existing single-factor authentication mechanisms do not suffice the security requirements.

Besides, the existing advanced security solutions are expensive and complicated, which makes them inappropriate for use widely in homes and small organizations. There is a need for a cost-effective, reliable, and user-friendly system that can be able to improve security through multi-authentication layers.

Hence, the problem statement for this project is to design and implement a smart door locking system that can alleviate the challenges and shortcomings of conventional and single-factor door locking systems by combining several authentication techniques into one smart and workable solution.

1.4 Aim and Objectives

The primary aim of this project is to design and implement a smart door locking system that enhances security through multi-factor authentication using biometric and RFID hardware components.

To achieve this aim, the following objectives have been identified:

- To develop a system that integrates face recognition, RFID and fingerprint authentication.
- To utilize Raspberry Pi 5 as the main controller for processing and decisionmaking.
- To implement a use-friendly interface using LCD display, LEDs and a buzzer for system feedback.
- To control access through a 12V electromagnetic door lock using a relay module.
- To ensure proper communication between all hardware components for smooth operation.

1.5 Scope of the Project

The scope of this project covers the following areas:

- Design and development of a prototype multi-factor smart door lock system intended for small-scale residential or office use.
- Integration of three different authentication methods: face recognition, fingerprint scanning, and RFID card reading, controlled by a Raspberry PI 5 edge module.
- Physical wiring configuration and testing all hardware devices: RC522 module, R307s fingerprint module, USB webcam, relay module and a 12V electromagentic door lock.
- Development of a local biometric enrollment tool for registering users across all three authentication methods
- Real-time feedback with 16x2 I2CLCD display, and a piezoelectric buzzer.
- Cloud telemetry using the MQTT protocol over a secure TLS connection to a HiveMQ broker that publishes access and system event logs in real time.
- System evalutaion conducted strictly under controlled environments.

The following areas are outside the scope of this project:

- Commercial deployment or large-scale installations.
- Development of a mobile application for remote access control.
- AI-based behavioural threat detection or adversarial attack resistance.
- Integration with external enterprise databases or third-party security infrastructure.
- Outdoor performance testing or testing under extreme lighting conditions.
- Backup power systems or redundancy mechanisms.

2. Background and Requirement Analysis

2.1 Introduction to Smart Security Systems

A Smart Security System is a contemporary security system that consists of electronic security gadgets connected to the internet. With a smart security system, the user can control his/her security gadgets using a smartphone application or an online service, making it easier to monitor, control, and manage the activities of the system. The system makes it easy to keep safe, secure, and be aware of what is going on around the house. (ION, 2025)

Features of the smart security system may include motion sensors, security cameras, alarms, smart locks, entry sensors, glass break sensors, keypads, key fobs, base stations, and doorbell cameras. All these components work together to ensure that suspicious activities are detected, alarms sent out, recording carried out, and that entry into the house can be controlled remotely. (ION, 2025)

Some of the additional features of the smart security system may be its compatibility with smart lighting, air conditioners, and even voice assistants, such as Google Assistant and Amazon Alexa.

2.2 Traditional Door Locking Systems

Conventional Door Locking Systems are traditional security systems which make use of mechanical locks and physical keys for securing doors. Such locks have existed since ancient times and are some of the oldest ways of securing homes and buildings from burglars. Traditional locks operate by means of mechanical parts like pins and tumblers that come together once the right key is placed inside. (Team, 2025)

Some of the most seen types of traditional locks include deadbolt locks, knob locks, and mortise locks. They require individuals to carry physical keys to access their properties.

Advantages of Traditional Door Locking Systems

- **Easy and Intuitive** : The conventional lock is simple to use since most individuals have knowledge of opening locks using physical keys.

- **Cheap Installation** : These types of locks cost less to install compared to the advanced digital locks used in modern homes.

Drawbacks of Conventional Door Locking Mechanisms

- **Less Convenient** : Individuals must always be carrying their keys, which may be lost, misplaced, or stolen.
- **Susceptible to Lock Picking** : There are some conventional lock mechanisms that are more susceptible to lock picking techniques and even bumping.
- **Less Secure** : These locks lack some advanced features like remote access, automatic locking, security alarms, and CCTV surveillance.
- **Long-Term Expenses** : Though the initial expenses are low, in the long run, there will be additional expenses associated with lock changing or duplicate key making among other security measures.

2.3 IoT-Based Security Systems

An IoT-Based Security System is an intelligent security system that makes use of the Internet of Things (IoT) technology to interconnect security equipment like surveillance cameras, smart locks, alarms, motion sensors, smoke detectors, and doorbells over the internet. All these interconnected gadgets help provide real-time monitoring, management, and protection of the respective building. (Rupareliya, 2025)

An IoT-based security system allows individuals to have remote control over their home security through their mobile phones, tablets, and personal computers. The equipment sends and receives data through the internet to enable automation, instant notifications, live video streaming, access control, and intelligent monitoring.

Typically, such security systems have various functionalities like:

- Camera systems that can monitor live activities and movements
- Biometric/smart locks that enable keyless access
- Sensors for detecting any movements or entries
- Alarms and sirens to trigger an alarm in emergencies

- Door-entry video systems that recognize visitors
- Fire and smoke sensors for safety checks
- Switches and smart lights

One of the key benefits of IoT security systems is that they are convenient and allow real-time monitoring. The user can operate the locks, access video footage, communicate with people at the door, and receive notifications all through the mobile application. AI technology can also be incorporated into such systems.

2.4 Multi-Factor Authentication Systems

Multi-Factor Authentication (MFA) is a security system specifically designed to validate the identity of a user through the requirement of two or more unique factors before allowing access into the system. MFA represents an improvement over the conventional Single-Factor Authentication (SFA) security systems, which mainly rely on one easily hackable knowledge-based factor such as passwords. The layered nature of MFA significantly reduces the chances of any malicious breaches into the system. (Ometov, et al., 2018)

For ensuring total independence in structure and cryptography, a good MFA system design involves employing measures that cut across at least two of the following .

- Knowledge Factor (Something You Know): Data stored within the user's memory. It is usually represented by constant alphanumeric sequences like passwords, PIN codes, or predefined security questions-and-answers
- Possession Factor (Something You Have): A tangible item or digital artifact that is owned by only one individual. The most popular examples of such factors are hardware tokens or smart cards, cryptographic keys (FIDO2 tokens), or mobile apps that generate TOTP
- Inherence Factor (Something You Are): Biological or behavioural characteristics inherent to the human being. This factor presupposes the use of biometric technology that includes finger printing, two- or three-dimensional analysis of facial features, iris scanning, or speech recognition.

2.5 Existing System Limitations

Despite the progress made by IoT-based and biometric security systems, several major limitations still exist in the solutions available today.

- **Single-Factor Dependency:** Most residential and small-scale security systems rely on a single method of authentication, such as a PIN code, an RFID card, or a physical key. This creates a single point of failure. If one of these methods is compromised, stolen, or replicated, the entire system can be bypassed without any additional protection.
- **High Cost of Advanced Systems:** Commercial-grade multi-factor authentication systems that use biometrics, such as facial recognition or fingerprint scanning, are mainly intended for business or government use. Their high costs for purchase, installation, and maintenance make them out of reach for individual homeowners or small organizations.
- **Lack of Remote Monitoring:** Many traditional and entry-level smart lock systems do not offer real-time event logging or remote monitoring features. This means administrators or homeowners cannot see access attempts, whether they are successful or denied, unless they are there in person.
- **Limited Fault Tolerance:** Current single-sensor systems lack a backup mechanism. If the primary authentication device malfunctions, such as a fingerprint sensor failing due to a dirty surface or a damaged RFID reader, the whole access control system stops working until someone addresses the issue manually.
- **Privacy and Security Concerns:** Systems that depend on cloud-based biometric processing send sensitive user information over the internet. This raises risks related to data interception, unauthorized access, and how third parties handle data. The lack of end-to-end encryption in many consumer-grade products makes these vulnerabilities even worse.

These limitations emphasize the need for a cost-effective, locally processed, multi-factor authentication system that provides strong security without sacrificing reliability or ease of use.

2.6 Requirement Analysis

This chapter will examine the structural requirements for implementing and managing the MFA IoT smart door lock solution. The process flow, performance metrics, and software/hardware prerequisites will be described in detail.

2.6.1 Functional Requirements

Functional Requirements refer to those activities, behaviour or operations that the system is expected to perform.

- **Multi-Modal Authentication:** The system will authenticate users by capturing their identity using three independent credentials streams namely; biometric face recognition, R307S fingerprint scanner and RC-522 RFID card reader. (Ometov, et al., 2018)
- **Actuator Control (Lock Management):** After successfully authenticating any of the valid and enrolled credentials, Raspberry Pi 5 should deliver a digital HIGH/LOW signal to the relay 5V module to switch off or on thereby unlocking the door using 12V electromagnetic door lock.
- **Real-time Status Output:** The system is expected to display real-time status states on the 16x2 I2C LCD screen such as access granted, access denied and scan card among others. In addition, the system will turn green light on when there is a success whereas the red light will be on during failures.
- **Audio Alerts:** The audio alerts generated by the system should be unique frequency patterns produced by the buzzer for differentiation of occurrences and immediate validation during the authentication process.
- **Internet-of-things Data Communication:** The communication system used in the process is to ensure that authentication records, times of entry, and system alerts are sent to IoT subscribers using a telemetry structure.

IoT Data Publishing: The system is expected to use the MQTT message protocol to publish data regarding authentications, time stamping and alerts.

2.6.2 Non-Functional Requirements

The non-functional requirements define the performance measures, quality parameters, and constraints of the system.

- Performance: End-to-end latency from biometric authentication to relay activation should be less than or equal to 2.0 seconds.
- Reliability and Fault Tolerance: In case one peripheral sensor (such as the fingerprint scanner) malfunctions, the other two biometric methods (RFID and Face Recognition) should remain functional independently of each other, ensuring that the controller's event handling process does not terminate.
- Security: Biometric credentials, local biometric arrays, and UID maps stored in the embedded memory of the edge node must be secured against any local unauthorized access and tampering to ensure user privacy. (Jain, et al., 2025)
- Usability: The interface feedback provided by the LCD screen, LEDs, and buzzer must be sufficiently simple for a novice user to understand their status without any training.
- Networking: The application layer protocol for data exchange should be based on the publish-subscribe model to reduce network overheads associated with telemetry transmission through low-bandwidth local networks.

3. System Design

3.1 System Overview

The proposed system is a smart door access solution that combines multiple authentication methods into a single integrated setup. Instead of depending on a single form of verification, this system allows access through multiple access methods such as fingerprint, face recognition and RFID cards. This multi-factor approach helps improve both security and flexibility for the users.

The core of the system that acts as the main controller and the decision-making unit is Raspberry Pi 5. It is responsible for determining whether access should be granted or denied. The USB webcam is used to take facial images, which are then processed for facial recognition. The RC-522 RFID module reads card data, while the R307S fingerprint sensor captures and matches fingerprints.

The inputs from the modules are handled by Raspberry Pi, which runs the necessary software to compare the data received with user information. If any of the authentication methods provide a valid, registered input; the Raspberry Pi sends the signal to the relay which then unlocks the 12V door lock module.

To ensure the design remains user-interactive, the system also includes a 16 x 2 LCD display that shows messages such as access granted, access denied or other instructions for the user. The LEDs provide quick visual feedback using green and red LEDs to signal unlocked and locked states. Audio feedback is given using a piezoelectric buzzer that produces different alerts during different system events.

Overall, the system is designed to ensure smooth communication between the hardware components while remaining reliable. By combining multiple authentication and feedback mechanisms, the system provides a more secure and intuitive access control solution.

3.2 System Design and Architecture Diagram

Circuit Designer

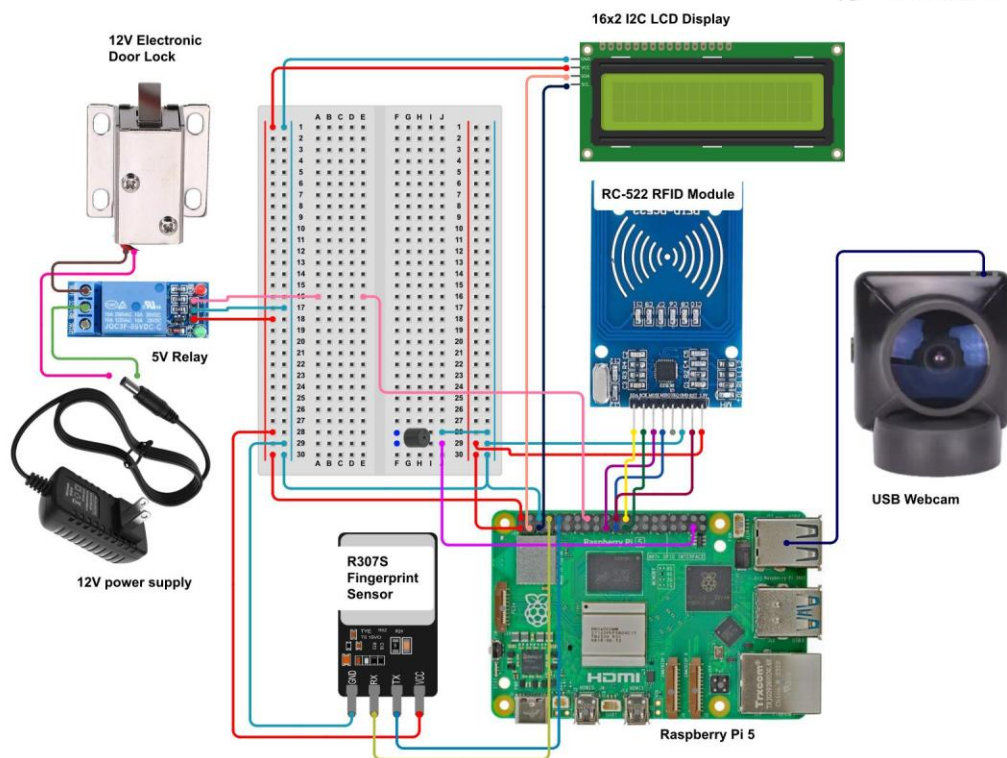


Figure 1 : System Architecture Diagram for the smart door-lock system

3.3 Flowchart of the System

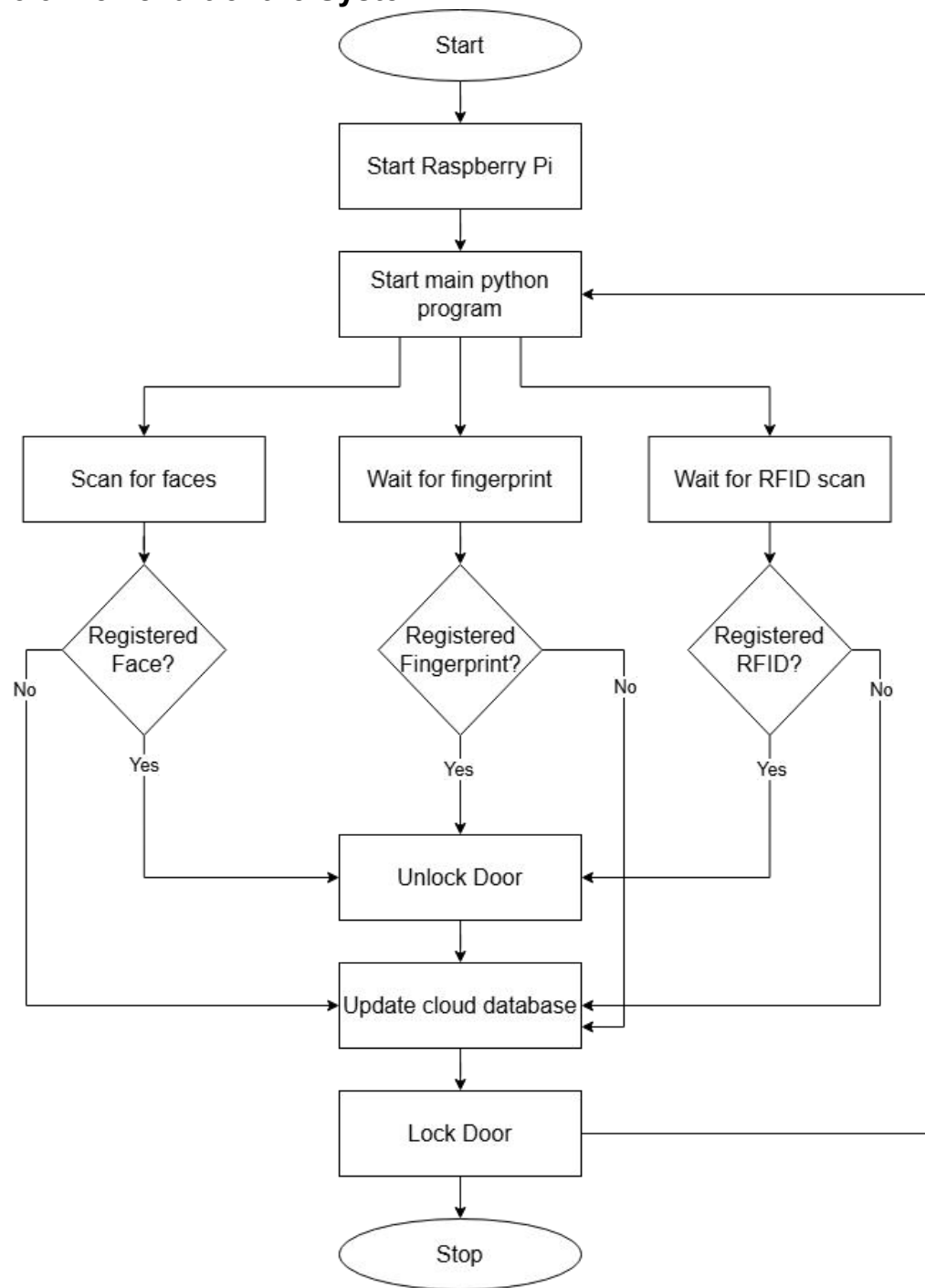


Figure 2 : Flowchart for the smart door lock system

3.4 Hardware Components

3.4.1 Raspberry Pi 5

The raspberry pi 5 is the main compute and control module. It can run complex and resource intense face detection algorithms combined with easy-to-use GPIO pins that can be configured by programming. It also supports U-ART, I2C and SPI protocols required by the attached sensors and modules. (Mind, 2026)

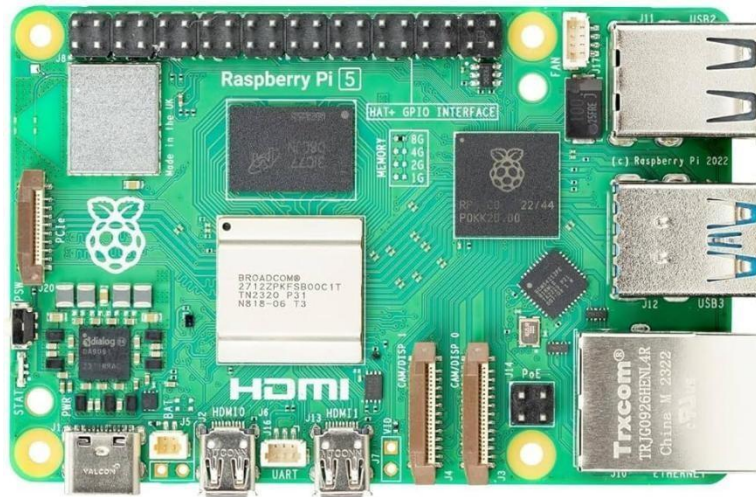


Figure 3 : Raspberry Pi 5 8GB

3.4.2 USB Webcam

The USB webcam will act as the sensor module to collect pictures of its surroundings; Raspberry Pi can then analyze the images for recognized or unrecognized faces. (Yealink, 2026)



Figure 4 : A 4MP USB Webcam

3.4.3 RC-522 RFID Module

The RC-522 module is required to scan and collect data from the RFID/NFC cards. It uses the SPI protocol to communicate with the Pi. (Jain, 2025)



Figure 5 : RC-522 RFID Module

3.4.4 16×2 I2C LCD Display

The LCD module acts as a user-interactive display to prompt the user and inform the user about the processes happening in the background. It uses the I2C protocol to communicate with the Pi.



Figure 6 : 16 x 2 I2C LCD display

3.4.5 R307S Fingerprint Sensor

The R307S module is used to add fingerprint-based authentication to the system. It captures and stores fingerprint data, which can later be used to verify users. The module connects to the Raspberry Pi using UART communication, making the identification process quick and reliable.



Figure 7 : R307S Fingerprint module

3.4.6 Piezoelectric Buzzer

The buzzer is included to give audio feedback during operation. It produces different sounds to indicate actions such as successful access, denied entry, or system warnings. It is controlled directly through the Raspberry Pi's GPIO pins.



Figure 8 : Piezoelectric buzzer

3.4.7 12V Electromagnetic Door Lock

The electromagnetic lock is responsible for physically securing the door. It stays locked when power is supplied and unlocks when the power is cut. This makes it suitable for controlled access systems where the lock is managed electronically.



Figure 10 : 12V Electromagnetic Door lock

3.4.8 5V Relay Module 2 Channel Relay Module

The relay works as a switch to control the door lock using the Raspberry Pi safely. Pi cannot handle higher voltages directly, the relay acts as a low-power control circuit for the 12V system.

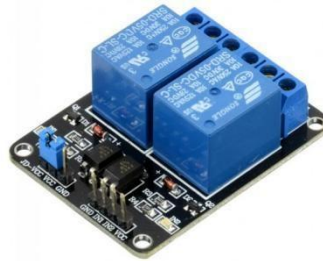


Figure 11 : 5V relay module

3.5 Software and Tools

3.5.1 Thonny IDE

Thonny IDE is used as the main development environment for writing and testing Python code on the Raspberry Pi. It provides a simple interface that is suitable for beginners and helps in debugging the system easily. (Thonny, 2026)

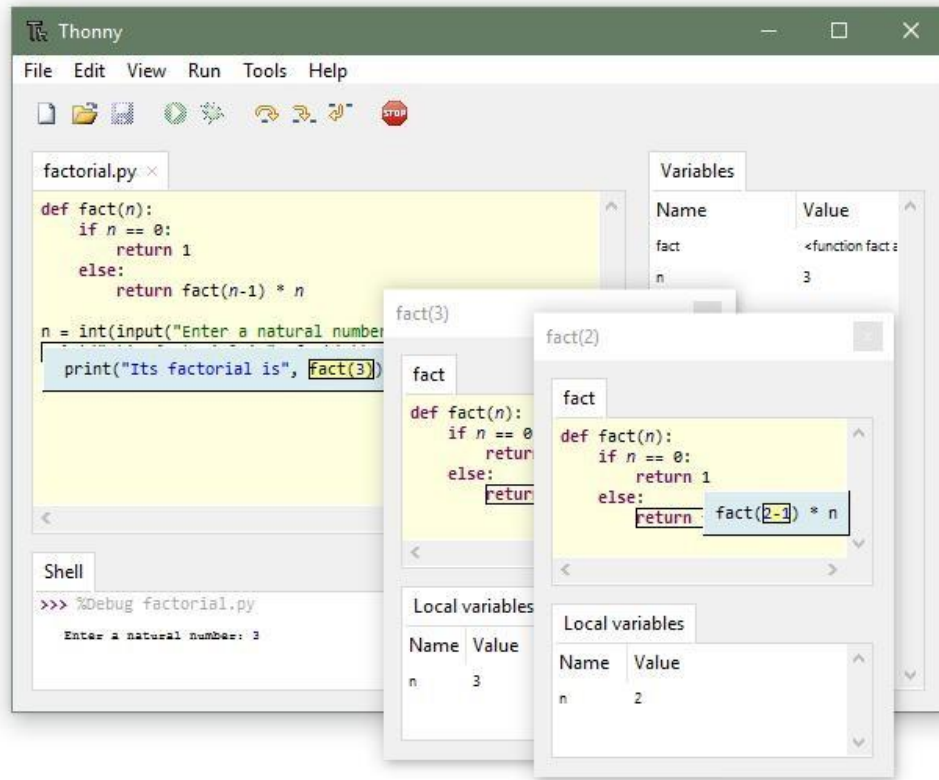


Figure 12 : Thonny IDE

3.5.2 Python 3

Python 3 is the primary programming language used to develop the system. It is responsible for handling logic, controlling hardware components, and integrating different modules together. (Zaripov, 2024)



Figure 13 : Python3 logo

3.5.3 OpenCV Library

OpenCV is used for image processing and face detection. It allows the system to capture images from the webcam and identify whether a face is recognized or not.

(GeeksforGeeks, 2025)



Figure 14 : OpenCV Logo

3.5.4 NumPy

NumPy is used to handle numerical data and arrays, which are essential when processing images in OpenCV. It helps improve performance and simplifies complex calculations.

(NumPy, 2025)



Figure 15 : NumPy Logo

3.5.5 RPi.GPIO

RPi.GPIO is used to control the Raspberry Pi's GPIO pins. It allows the system to interact with hardware components like LEDs, buzzer, and relay module. (Electronics,

2025)



Figure 16 : RPi.GPIO python library

3.5.6 MFRC522 Library

This library is used to interface with the RC-522 RFID module. It enables reading data from RFID cards and integrating it into the authentication system. (Aditya, 2026)

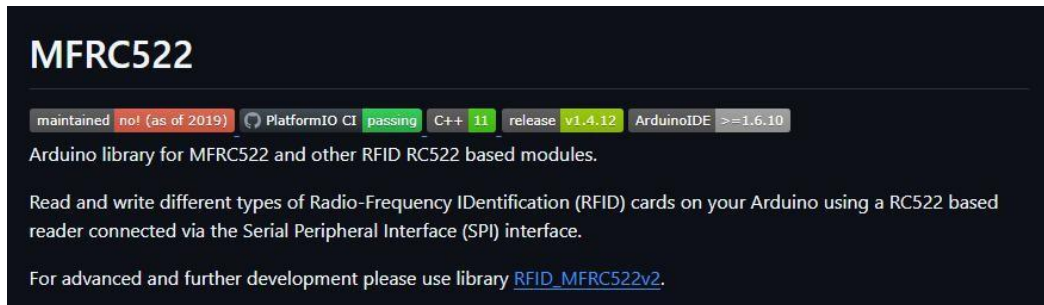


Figure 17 : MFRC522 python library.

3.6 Communication and Cloud Integration

3.6.1 MQTT Protocol

The MQTT (Message Queuing Telemetry Transport) is a very light-weight, open protocol, and a messaging standard made for the Internet of Things (IoT) and Machine to Machine (M2M) communication . (MQTT, 2024)

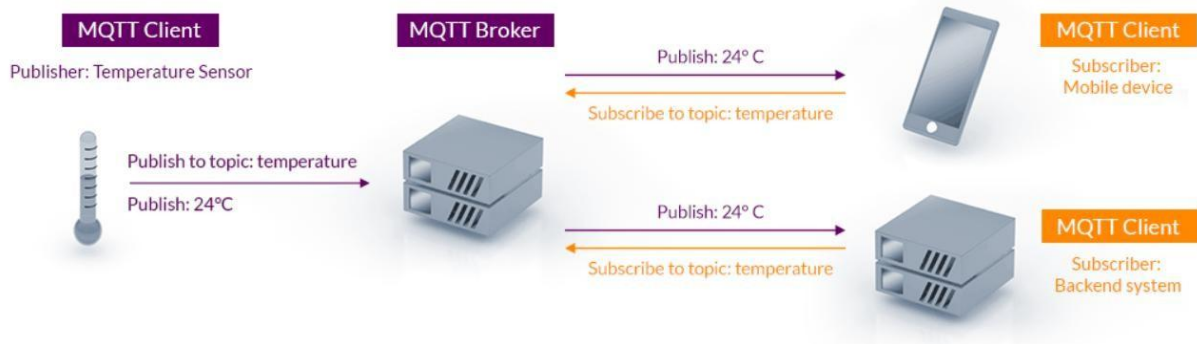


Figure 18 : MQTT Publish / Subscribe Architecture

3.6.2 HiveMQ Integration

HiveMQ is an enterprise-level MQTT broker engineered to connect IoT devices with cloud environments. Built on a masterless cluster architecture, it ensures continuous uptime and high fault tolerance while managing millions of concurrent data streams.

The table below outlines the formal network parameters required to interface with the provisioned cluster:

[← Back](#)
Free #1

[Overview](#)
[Access Management](#)
[Web Client](#)
[Getting Started](#)

Connection Details

Comprehensive details and statistics for your cluster

URL

d77552e1720b49d3a4bdea760d0a27a8.s1.eu.hivemq.cloud

Port

8883

WebSocket Port

8884

TLS MQTT URL

d77552e1720b49d3a4bdea760d0a27a8.s1.eu.hivemq.cloud:8883

TLS WebSocket URL

d77552e1720b49d3a4bdea760d0a27a8.s1.eu.hivemq.cloud:8884/mqtt

Figure 19: MQTT connection details.

Please connect to the WebClient in order to subscribe to topics

Connection Settings

Connect to your HiveMQ Cloud Cluster with your credentials. Or, connect instantly with autogenerated credentials.

Username

raspi

Password

.....

Connect

 or

Connect with autogenerated credentials

Figure 20: MQTT user login

Messages 0

Clear all messages



No messages

You don't have any messages yet.

Let's fix that! ✨ Subscribe to all messages via the # wildcard.

Subscribe to all messages

Figure 21: Accessible MQTT WebUI.

4. Development Process

The chapter below outlines an in-depth, technical discussion regarding the implementation process of the multi-factor IoT smart door lock system. It explains how the system was developed systematically from scratch to create the architecture described in chapter three of this study.

4.1 Development Methodology

This implementation phase was undertaken via the iterative prototyping method based on the Agile approach. Considering the high risk associated with the integration of several separate hardware components, such as communication interfaces and power control units, adopting a rigid waterfall method could pose considerable risks during integration at the end of the life cycle.

With iteration, each mode of authentication (RFID, biometrics, and facial recognition) underwent development, debugging, and unit testing as separate systems in its respective software code branch before being compiled. Through this method, any peripheral pin conflicts, timing issues, and Input/Output (I/O) problems would be addressed early on, thus minimizing fault isolation during the hardware/software integration process.

4.2 Stage 1 – Hardware Setup and Raspberry Pi Configuration

Stage one involved establishing the optimized, stable OS and software environment on the Raspberry Pi 5 single-board computer in preparation for performing high throughput edge controller operations.

First, the official Raspberry Pi OS (64-bit Debian Bookworm architecture) was installed into a high-performing class 10 MicroSD card by using Raspberry Pi Imager. 64-bit execution environment was chosen to allow the utilization of hardware-level optimization flags which are required for real-time computer vision processing (Emergent Mind, 2026).

Secondly, the peripheral devices' buses were set up through non-graphical system utility. The Serial Peripheral Interface (SPI) bus is used for connecting to the RFID transceiver, Inter-Integrated Circuit (I2C) bus is used for controlling the human-machine dashboard,

while hardware-level UART serial mapping was separated from Linux console to preserve raw serial communication lines for fingerprint scanner

Finally, to avoid package dependency conflicts and isolate system-level Python environment, the following terminal commands were used to initialize isolated virtual environment (venv): **python3 -m venv venv**

source venv/bin/activate

Within this sandbox, the project's dependency stack was installed using the Python package installer (pip). This process successfully imported lower-level GPIO abstraction layers (Rpi.GPIO), matrix array computation capabilities (numpy), computer vision libraries (opencv-python), neural-network model(yolo12s.pt), hardware access layering (spidev), and MQTT application layering clients (paho-mqtt)

4.3 Stage 2 – RFID Authentication Module Development

This development phase centered on the installation of the possession layer, which entailed connecting the 13.56 MHz RC-522 RFID transceiver with the primary controller through the SPI bus.

The hardware connection involved connecting the SCLK, MOSI, MISO, and CS pins on the module to the corresponding pins on the Raspberry Pi 5. An object-oriented driver Python program was developed employing the spidev and MFRC522 libraries to access the hardware registers.

The software logic operates on an asynchronous loop whereby a radio frequency request command is periodically sent to scan for ISO/IEC 14443A compatible contactless smart cards in the electromagnetic nearfield of the antenna (Jain, et al.,

2025) . Once a card enters this near-field range, the unique ID (UID) byte array is captured. The UID byte array from the Python program is stripped off of all erroneous formats, converted to hexadecimal format, and checked against a pre-defined list of valid UIDs stored in an array memory. Once an identity match occurs, the class returns an authenticate.

4.4 Stage 3 – Fingerprint Authentication Module Development

The hardware layer of physical inherence was implemented by embedding the optical biometric sensor, R307S, within a serial pipeline.

The R307S sensor alleviates processing overheads by doing all the computations involved in image acquisition, features extraction, and mathematical template matching in its inbuilt digital signal processor (DSP). The sensor relays the transaction results to the Raspberry Pi 5 through the standard UART asynchronous serial interface communication at a predefined hardware baud rate of 57600 bps.

This interaction is orchestrated by the implementation of packet wrapper structure in the driver script. Each command sent to or received from the sensor conforms to a defined byte protocol: the standard two-byte header (0xEF01), four bytes of module factory address, single byte of packet identification flag, two bytes of packet length, actual packet data, and two bytes of checksum arithmetic. The Python logic continuously checks the UART serial buffer for incoming acknowledgment bytes. On receiving user's finger over the optical prism, the R307S generates local fingerprint minutiae template, compares the result against the index stored in its internal flash memory, and sends back a result byte.

4.5 Stage 4 – Face Recognition Module Development

To meet the non-functional requirement of processing time of 2.0 seconds, an extremely optimized AI face recognition pipeline was implemented right onto the Raspberry Pi 5.

Rather than using the bulky, uncompressed Convolutional Neural Networks that max out the capacity of the CPU and result in input/output delay, the pipeline makes use of the YOLO NCNN (Next Generation Convolutional Neural Network).

This process involved an iterative loop in three stages:

- Image Acquisition: The camera with a resolution of 4MP is integrated with the USB interface and can ingest frame data into a video capture stream running in OpenCV at a constant rate of 30 frames per second (MDPI, 2024).

- **Face Detection/Vector Extraction:** Incoming image data undergo scaling to fit the input dimensions of the lightweight facial detection model. This involves detecting face coordinates and generating a 128D feature vector.
- **Classification Inference:** The generated face vectors are analysed via multiple matrix computations within the NumPy framework, and the resultant distances are compared to user profiles to determine the lowest value. When this value is lower than a predetermined threshold, the face is recognized without relying on cloud-based AI computing.

4.6 Stage 5 – LCD, LEDs and Buzzer Integration

Feedback systems that involve users' participation have been consolidated into one human-to-computer interface module controlled by simple GPIO status and bus messages.

To avoid using many pins in total for the Raspberry Pi 5 header, a 16x2 characters LCD screen has been used with an I2C backpack circuit board. Thus, only two active buses (Serial Data and Serial Clock) are used to establish communication channels to send instructions.

A special helper class converts hexadecimal code into memory address changes within the LCD controller. Contextual string messages such as "Scan Card...", "Access Approved" or "Authentication Failed" are displayed on the LCD monitor. Finally, a special PWM-enabled pin is connected to the piezo buzzer which can generate quick beeps in case of successful card scans or alarm signals if the user enters wrong credentials multiple times in row.

4.7 Stage 6 – Electromagnetic Door Lock and Relay Control

Electrical isolation was used as a physical access control management system to ensure the protection of the main SBC.

Since the 12V electromagnetic door lock consumes much current which will inevitably fry the low-powered 3.3V digital logic lines of the processor, there was placed a relay module

with electrical isolation between these two systems powered by an optocoupler and 5V voltage source.

The program will set the status of the special pin on the SBC as ACTIVE and thus will power an optoelectronic LED in the relay module that activates a photosensitive receiver which then switches off a switch. It disconnects the connection with the external 12V line that supplies electricity for the magnetic lock which allows opening the lock for five seconds before returning the relay to its initial status and closing the door again.

4.8 Stage 7 – MQTT Communication Setup

The data refreshes and system event logs are sent from the edge device via an isolated network publisher loop.

The implementation uses the MQTT protocol, which employs the paho-mqtt client library for packet communication. The script launches a non-blocking asynchronous network client loop running in the background.

Each time the event occurs, irrespective of the granting or denying of access, the data layer creates a JSON-formatted payload consisting of the node identifier, an accurate epoch timestamp, the exact type of authentication factor utilized (RFID, Fingerprint, or Face), and the outcome of the transaction. It is then published on the topic hierarchy (e.g., door/auth) using a QoS level of 1, ensuring reliable packet transmission through various local network conditions.

4.9 Stage 8 – HiveMQ Cloud Integration

Initially, a HiveMQ cluster was set up as the main cloud infrastructure. The network architecture was designed to expose specific, secure ports to enable encrypted communication channels. This centralized HiveMQ broker acts as the single telemetry entry point. It directly gathers real-time access logs and receives messages sent from the remote Raspberry Pi 5 edge node that controls the physical door lock.

Besides handling incoming telemetry, the system uses HiveMQ's bidirectional features to send remote execution commands back to the edge node. Control messages and system instructions, such as triggering temporary door unlocks for certain durations, updating

access permissions, or forcing remote reloads of software modules, come from the central infrastructure. These commands travel securely through targeted HiveMQ command topics down to the Raspberry Pi 5 node. This setup allows for realtime remote configuration and immediate physical access overrides.

In addition to message routing, HiveMQ manages the complete processing of these incoming and outgoing messages. The broker evaluates raw JSON telemetry data through custom extension logic and integrations. It dynamically updates operational metrics, maps access frequency tables, and automatically sends external email or text alerts if it detects consecutive unauthorized access failures at the door.

4.10 Final System Integration

The integration stage brought together the eight independent sub-modules into a single, seamless, non-blocking operational loop.

To avoid blocking processes such as inference on faces and delays caused by reading from the RFID card, the structure of the program was revamped to operate in a nonblocking loop model using OOP in Python.

In the primary code file, each individual peripheral is initialized in its own independent runtime segment. The event manager component inspects the incoming authentication credentials. In case of any positive identification by the tracking subsystem, the physical relay control component is activated, and the feedback screen is concurrently updated.

Lastly, a non-blocking thread for logging the events in the cloud is initiated.

5. Results and Findings

Verification and validation of the multi-factor IoT smart door lock solution will be covered in this chapter. To validate the performance of the combined hardware, edge computing, and cloud-based services of the IoT ecosystem in different operational environments, an extensive test matrix has been carried out.

5.1 Testing Overview

The testing approach adopted used a bottom-up validation technique, moving from the verification of individual units (Unit Testing) to the interface validation between different

functions (System Testing). In this way, each test case was aimed at assessing the system for compliance with the specified requirements outlined in section 2.6.

5.2 Unit Testing

5.2.1 RFID Module Testing

Table 1 : RFID Module Testing

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
RFID-01	Verify physical power delivery	Applying 3.3V rail from Raspberry Pi 5 to RC-522.	Onboard power LED illuminates steadily.	PASSED . Power rails stable.
RFID-02	Verify software polling state	Initialize the tracking script wrapper.	Program enters an unblocking loop, waiting for card proximity.	PASSED . SPI bus listening without overhead timeouts.
RFID-03	Verify physical proximity scan	Bring an ISO/IEC 14443A tag near the antenna.	Sensor antenna energizes the tag via magnetic induction.	PASSED . Physical communication established.
RFID-04	Verify data parsing and extraction	Pass the card directly over the reader's face.	Program captures, cleans, and outputs the unique identification (UID) string.	PASSED . Card UID detected and printed in the software terminal.



Figure 22: RFID module lights up when connected.

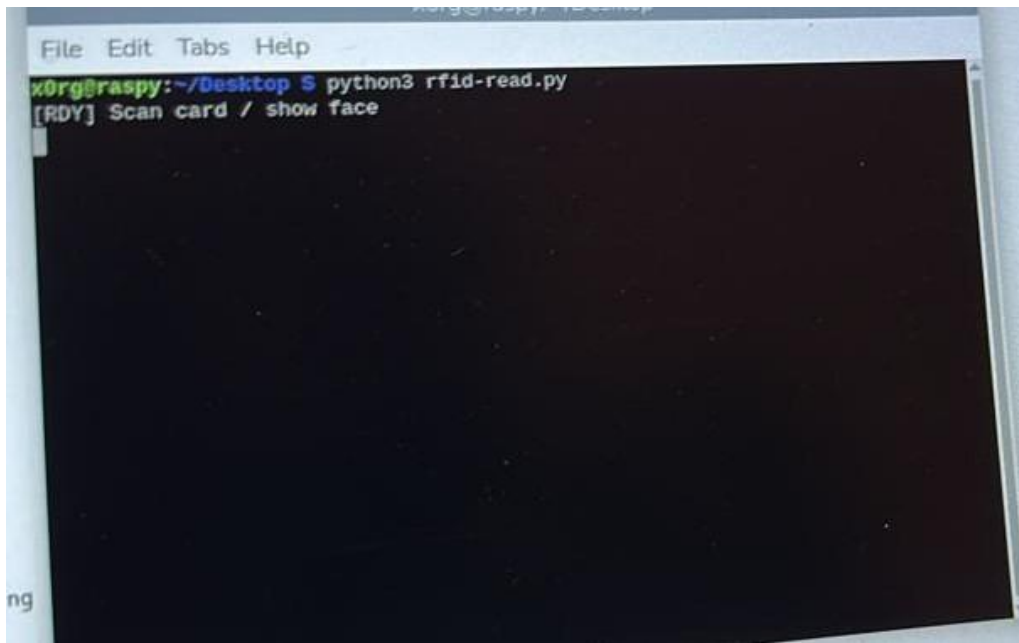


Figure 23: Running program to read RFID card.



Figure 24: Physically scanning RFID card.

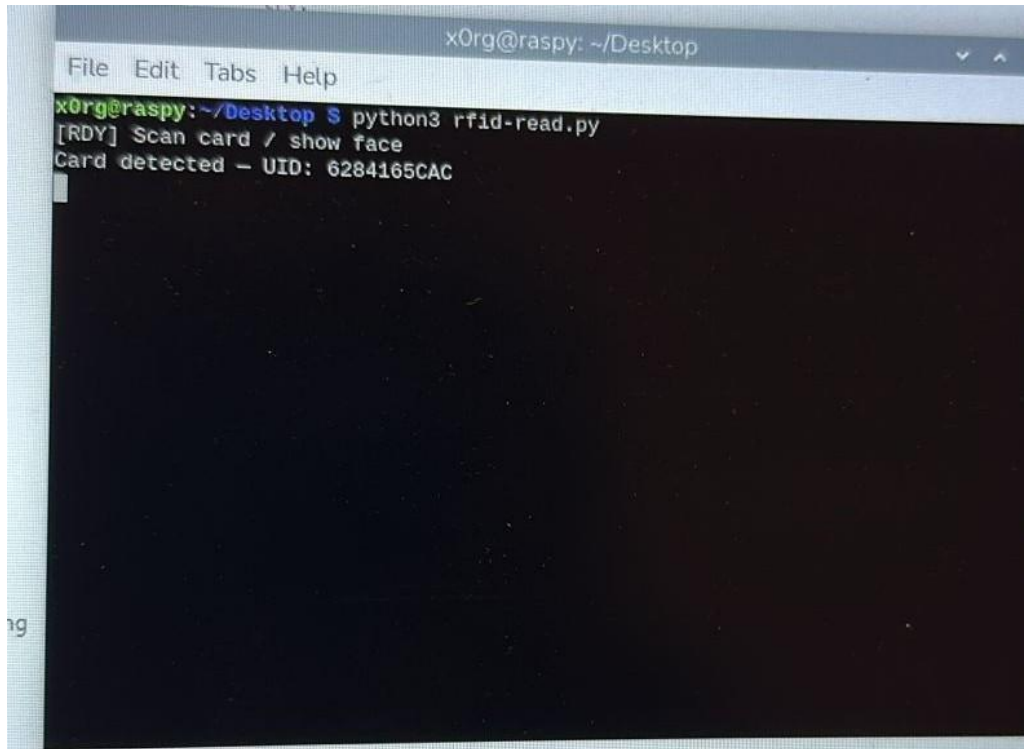


Figure 25: Card's UID appears and is successfully read.

5.2.2 Fingerprint Sensor Testing

Table 2 : Fingerprint Sensor Testing

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
FP-01	Verify hardware power and initialization	Connect 5V power and ground lines to R307S module.	Onboard optical prism illuminates, indicating sensor power stability.	PASSED. Visual LED initialization confirmed.
FP-02	Verify UART communication readiness	Run the biometric backend framework in Thonny.	Serial link opens at 57600 bps; device handshake returns code 0x00.	PASSED. Communication link ready for ingestion.

FP-03	Verify physical image capture	Place a finger squarely on the optical sensor prism.	Prism detects surface moisture/ridges and triggers internal capture matrix.	PASSED. Finger scanned and imaged.
FP-04	Verify template extraction and matching	Scan a previously enrolled fingerprint profile.	DSP extracts minutiae, matches template index, and outputs success byte.	PASSED. Fingerprint detected and validated successfully.

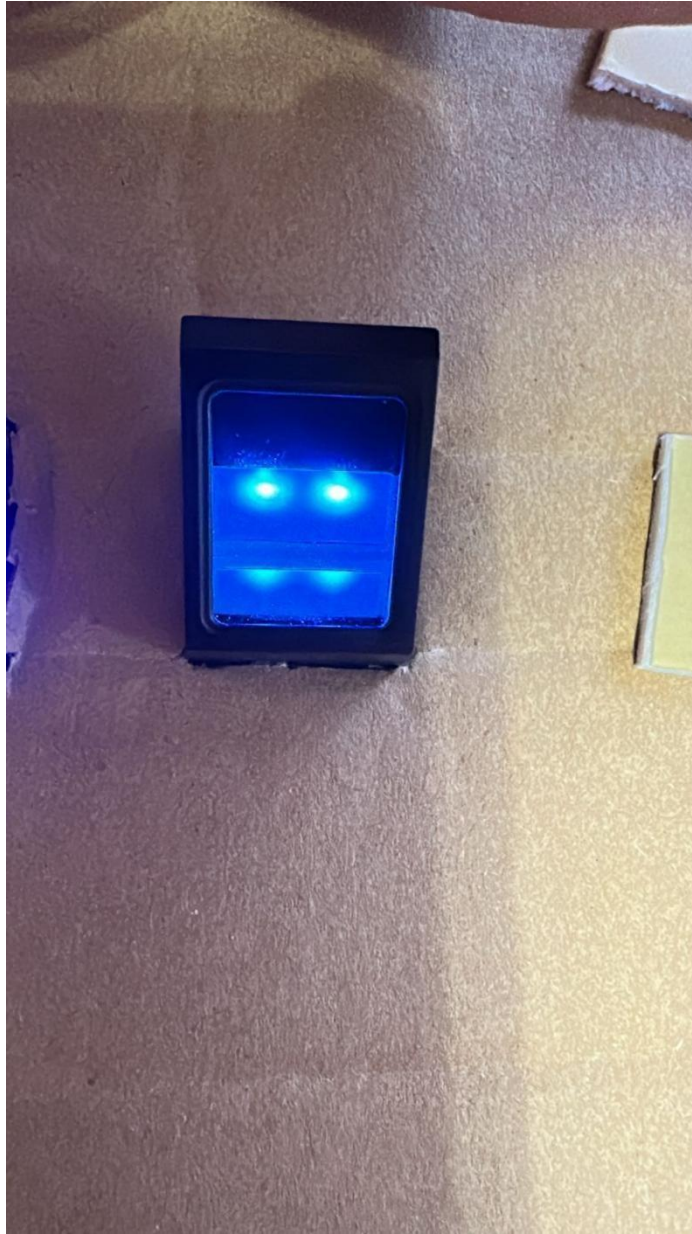
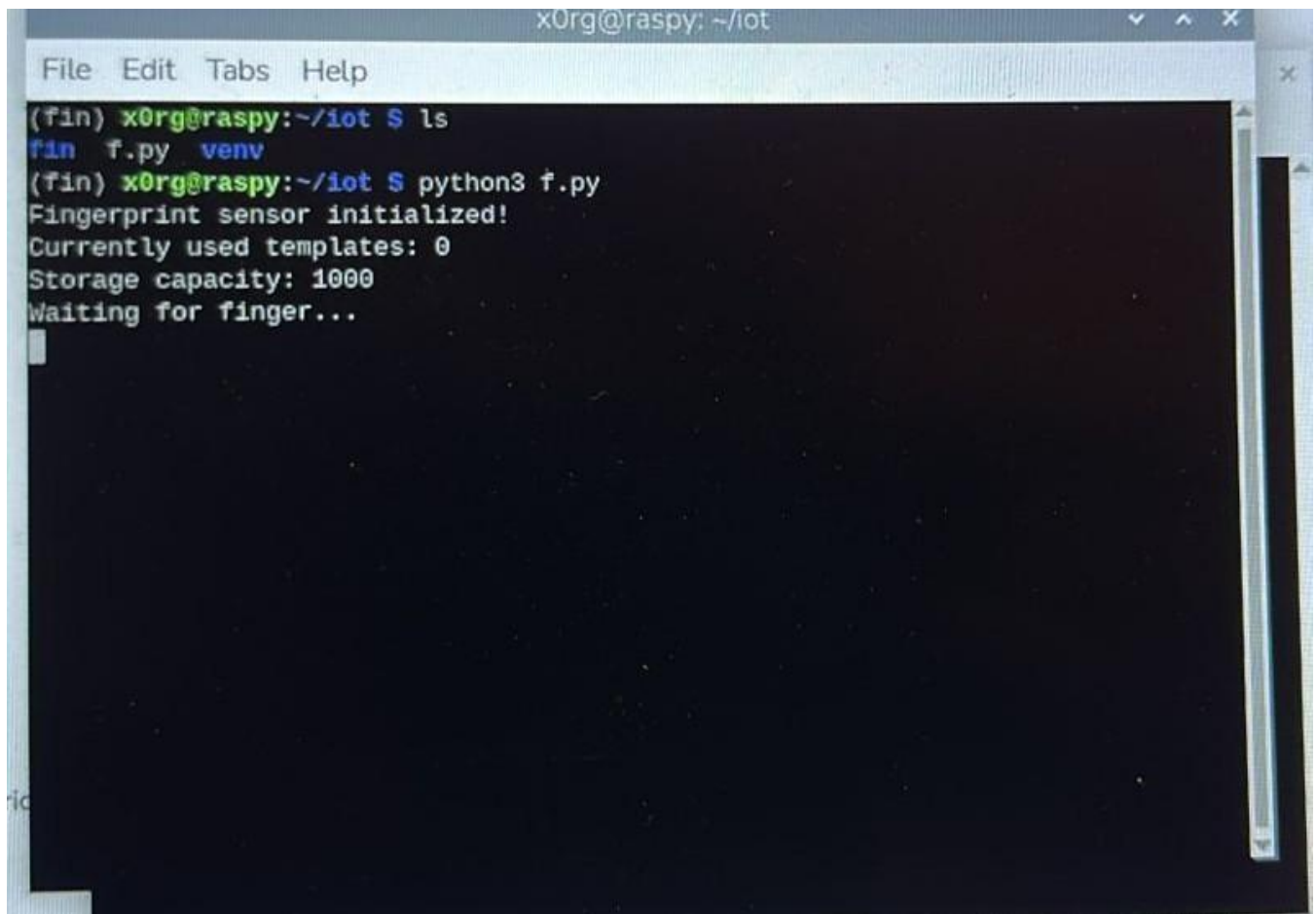


Figure 26: Fingerprint module turns on when powered.



A terminal window titled 'x0rg@raspy: ~/iot' with a menu bar (File, Edit, Tabs, Help). The terminal shows the following commands and output:

```
(fin) x0rg@raspy:~/iot $ ls
fin f.py venv
(fin) x0rg@raspy:~/iot $ python3 f.py
Fingerprint sensor initialized!
Currently used templates: 0
Storage capacity: 1000
Waiting for finger...
```

Figure 27: Program ready to read and test fingerprint.

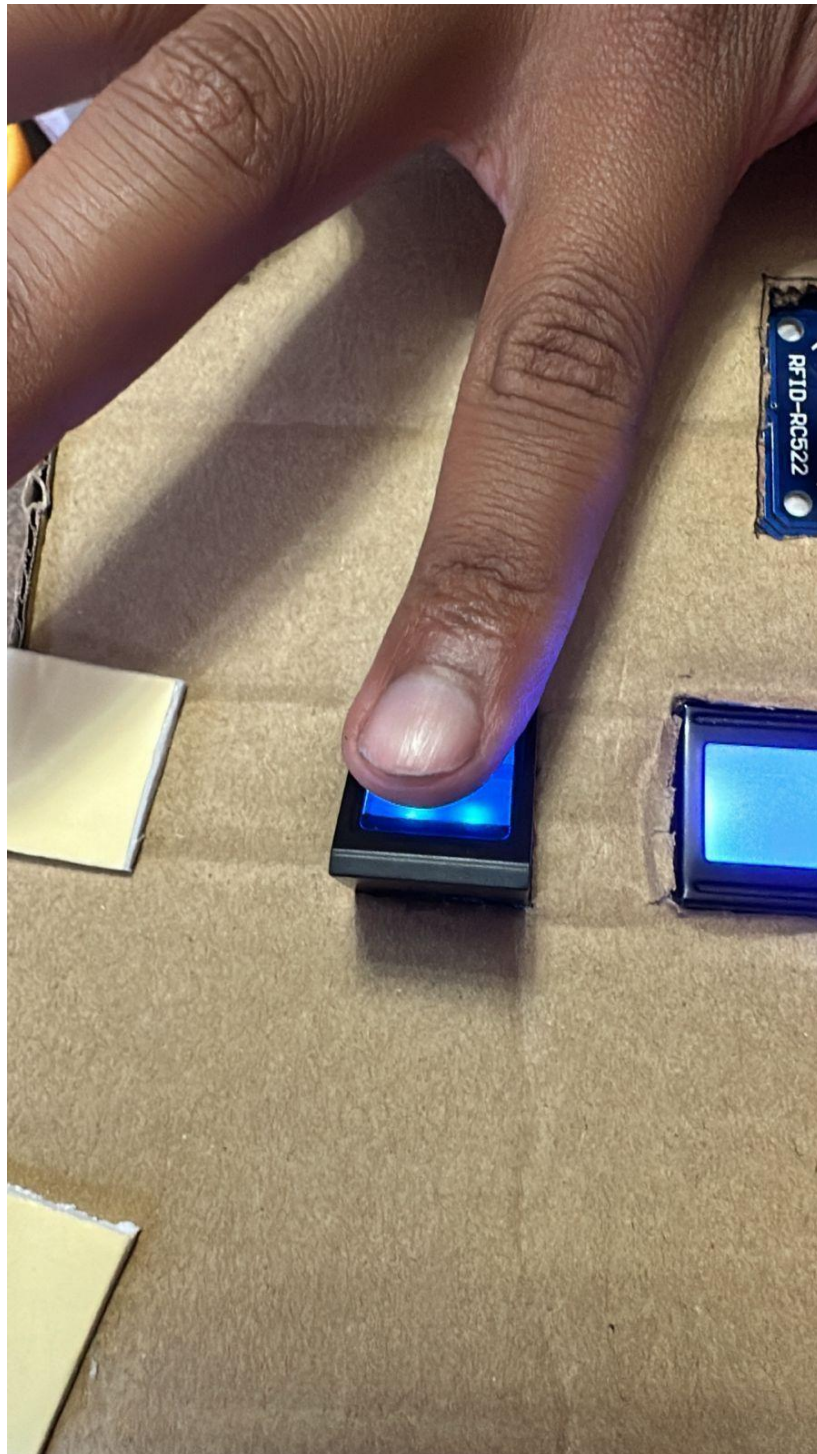
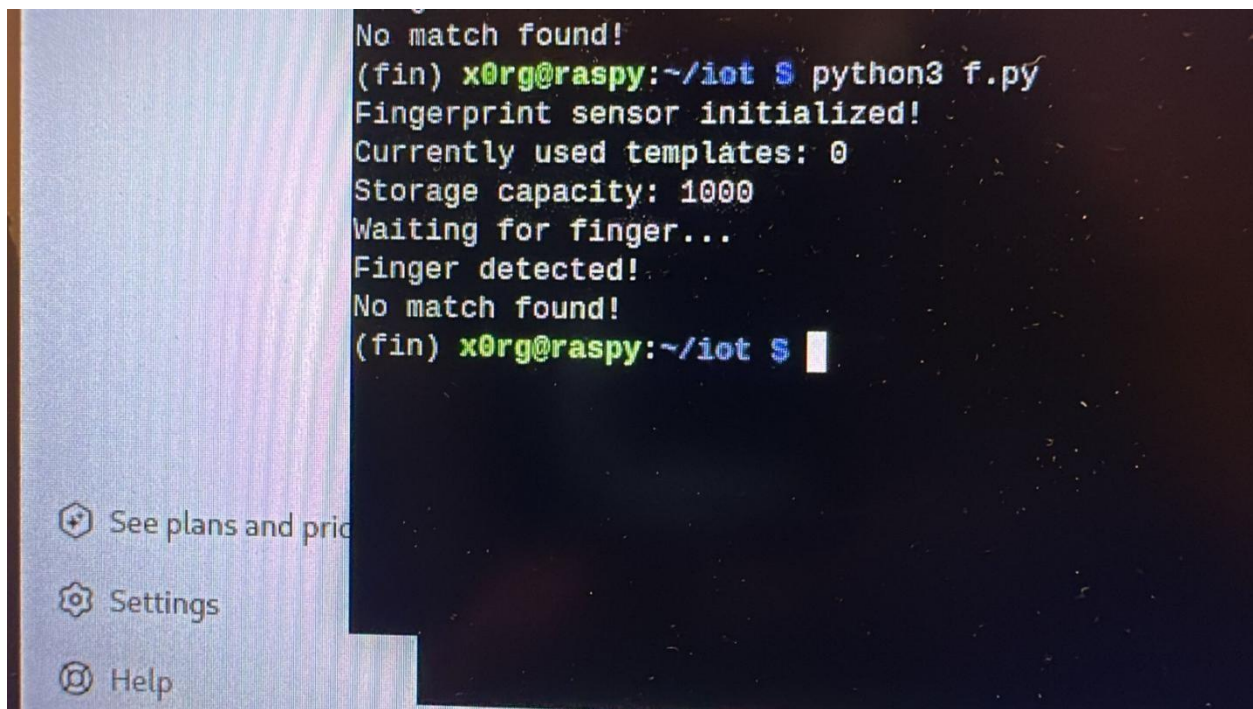


Figure 28: Physically scanning fingerprint.

A terminal window on a Raspberry Pi. The prompt is (fin) x0rg@raspy:~/iot. The user has run python3 f.py. The output shows the fingerprint sensor is initialized, there are 0 templates, storage capacity is 1000, and it is waiting for a finger. A finger is detected, but no match is found. The prompt is now (fin) x0rg@raspy:~/iot. To the left of the terminal, a sidebar is partially visible with icons for 'See plans and price', 'Settings', and 'Help'.

```
No match found!  
(fin) x0rg@raspy:~/iot $ python3 f.py  
Fingerprint sensor initialized!  
Currently used templates: 0  
Storage capacity: 1000  
Waiting for finger...  
Finger detected!  
No match found!  
(fin) x0rg@raspy:~/iot $
```

Figure 29: Fingerprint is detected but is not registered.

5.2.3 Face Recognition Testing

Table 3 : Face Recognition Testing

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
FACE-01	Verify video frame capture	Initialize OpenCV streaming pipeline from USB webcam.	4MP webcam initializes, providing raw video frames at a locked 30 FPS.	PASSED. Video capture interface active.
FACE-02	Verify local facial localization	Present an individual face within the camera's field of view.	Tencent NCNN model bounds facial area, isolating the region of interest.	PASSED. Bounding box registers around face instantly.
FACE-03	Verify 128-D embedding extraction	Execute neural network inference on the target face bounding box.	Model outputs a precise mathematical array representing landmark distances.	PASSED . Descriptive embedding generated successfully.
UTFACE-04	Verify vector distance classification	Match extracted vector against authorized user NumPy array.	System computes Euclidean distance; values below threshold yield positive match.	PASSED. Face recognized locally in under 1.5 seconds.

5.2.4 Relay Module Testing

Table 4 :Relay Module Testing

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
--------------	----------------	------------------------	------------------	------------------------

RLY-01	Verify rest state electrical mapping	Power on system without active authentication credentials.	Relay remains unenergized; 12V circuit closed, keeping door lock extended.	PASSED. Failsecure state verified at rest.
RLY-02	Verify driver software tracking state	Initialize the main GPIO controller library.	Assigned digital pin configures to output mode, holding logic LOW.	PASSED . Initial control line state stable.
RLY-03	Verify dynamic state transition	Trigger a simulated software validation token.	Controller pushes logic state from LOW to HIGH on the relay pin.	PASSED. Digital pin voltage shifts correctly.
RLY-04	Verify actuator mechanical switching	Monitor the 12V electromagnetic lock mechanism.	Optocoupler activates, opening mechanical contacts to retract door lock latch.	PASSED. Latch releases immediately upon state change.

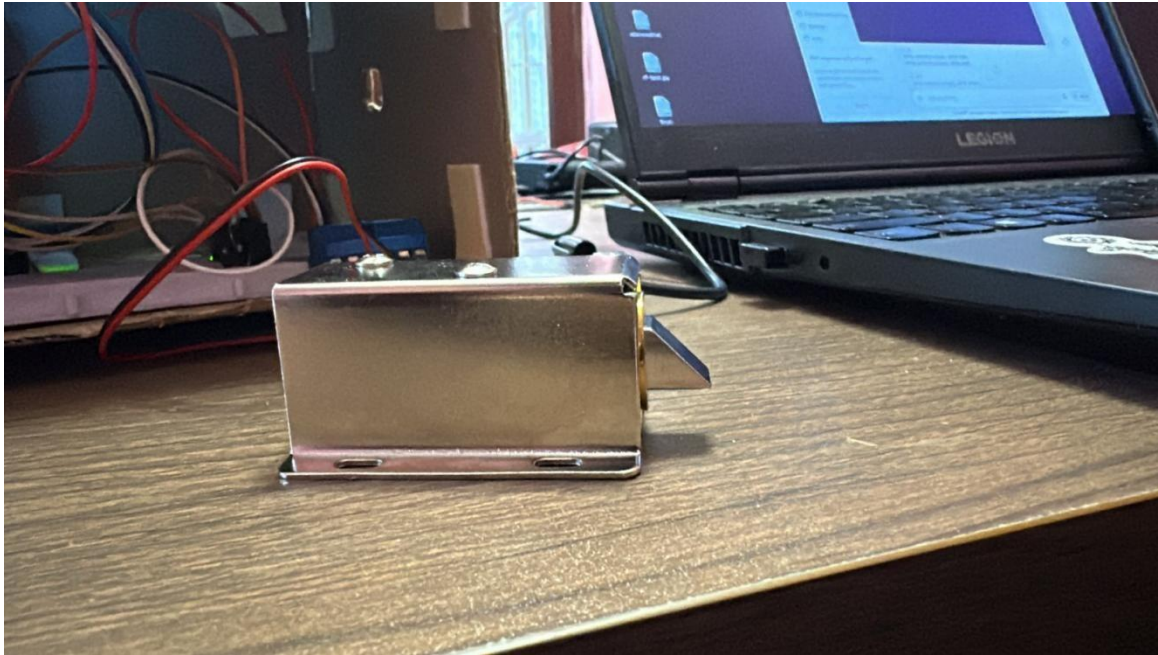


Figure 30: Door lock bolt is extended when unpowered.

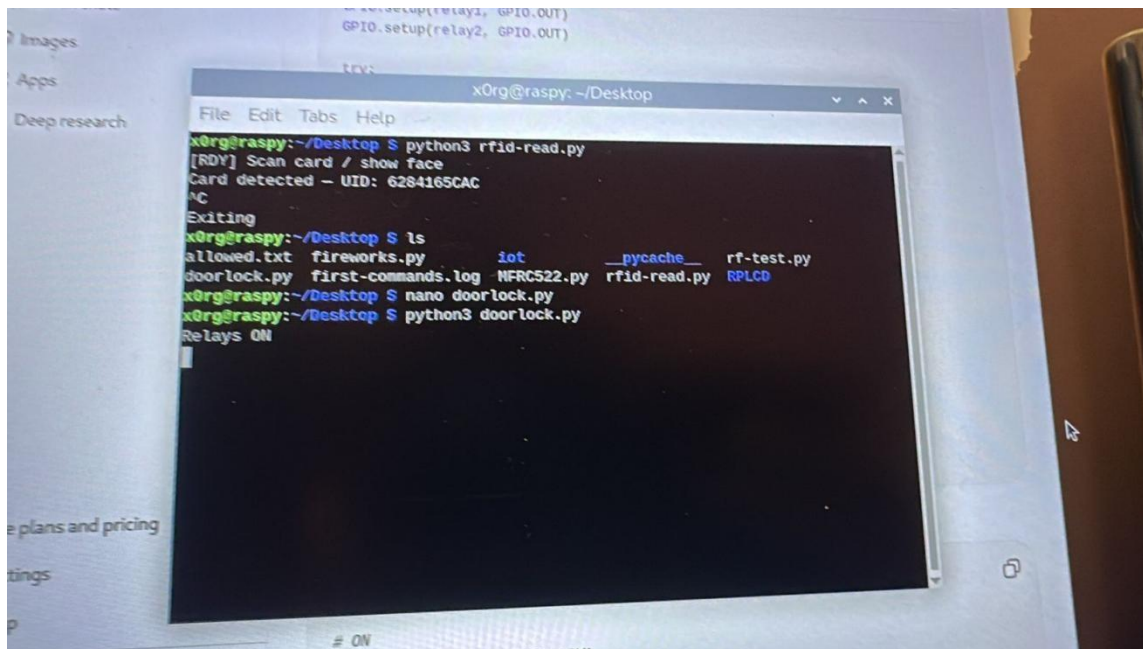


Figure 31: Program is run and the state is changed from OFF to ON

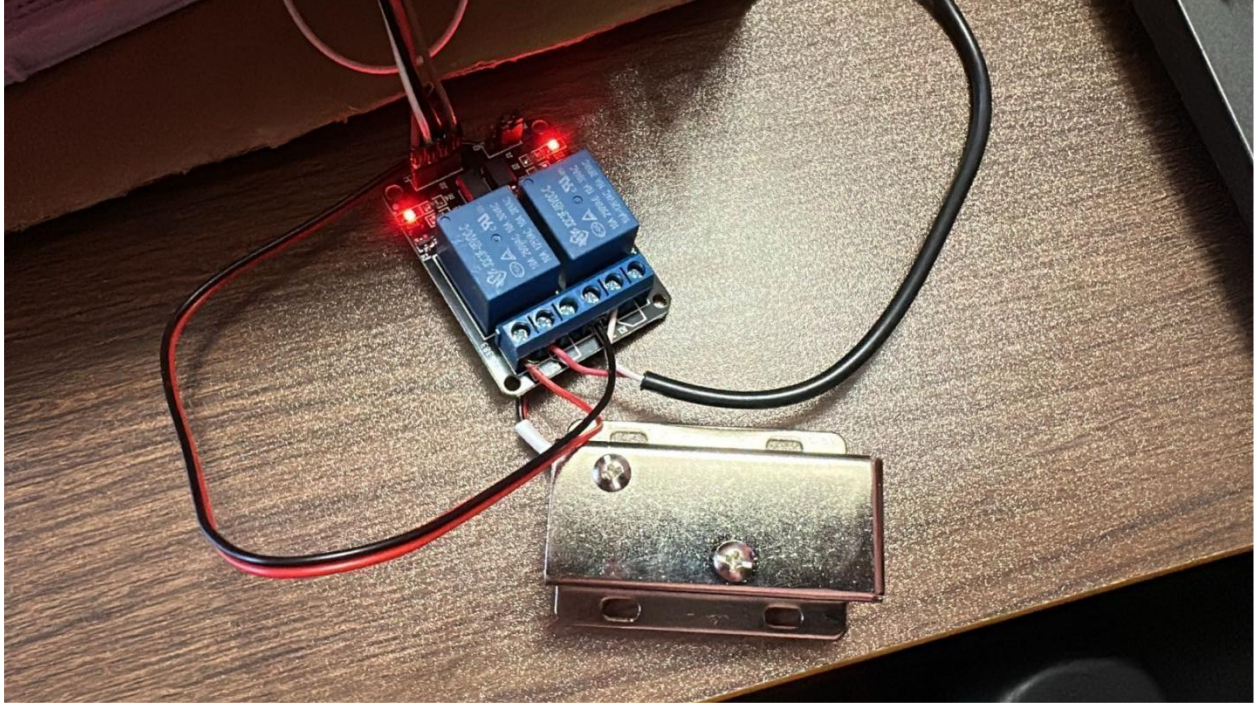


Figure 32: Relay module lights up and the bolt is retracted (success).

5.2.5 LCD and Buzzer Testing

Table 5 : LCD and Buzzer Testing

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
HMI-01	Verify LCD bus connection and power	Power on the I2C backpack interface rail.	LCD panel matrix initializes, displaying backlighting across lines.	PASSED. Panel powered and ready.
HMI-02	Verify output text handling readiness	Send string clear command to I2C register address.	Screen purges older strings, pointing cursor to position (0,0).	PASSED. Bus commands executing correctly.
HMI-03	Verify dynamic transaction rendering	Present an authorized credential factor to any sensor.	LCD instantly prints situational notification text like "Access Approved".	PASSED. String rendered clearly on the screen.
HMI-04	Verify sync acoustic warning trigger	Route parallel pulse-width modulation output to buzzer.	Piezoelectric buzzer emits highfrequency audio beep in sync with LCD updates.	PASSED. Audible alert delivered alongside visual text.



Figure 33: I2C LCD module powers up when connected.

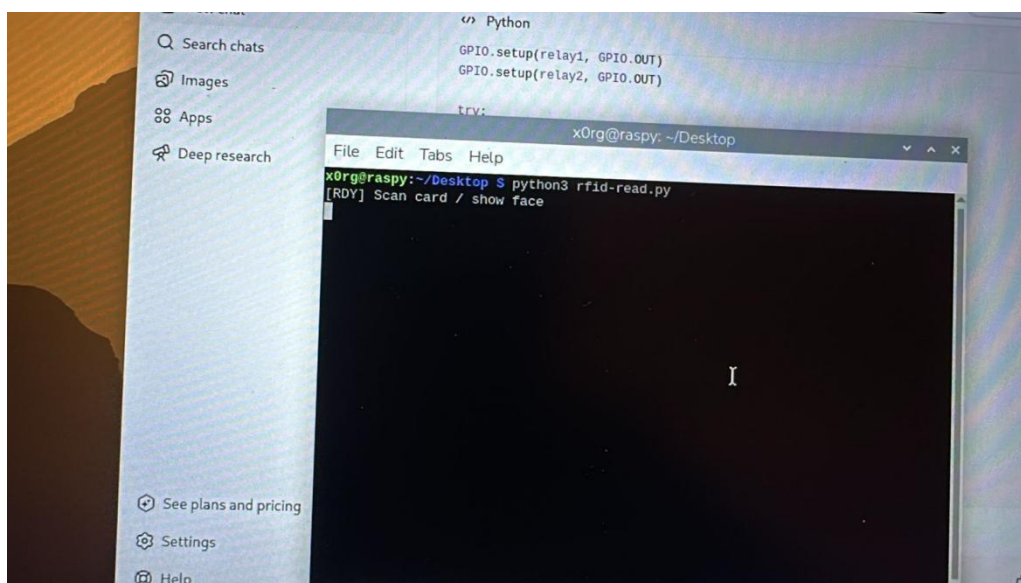


Figure 34: Program scans for RFID to display its UID.



Figure 35: I2C module displays the UID for the RFID (success).

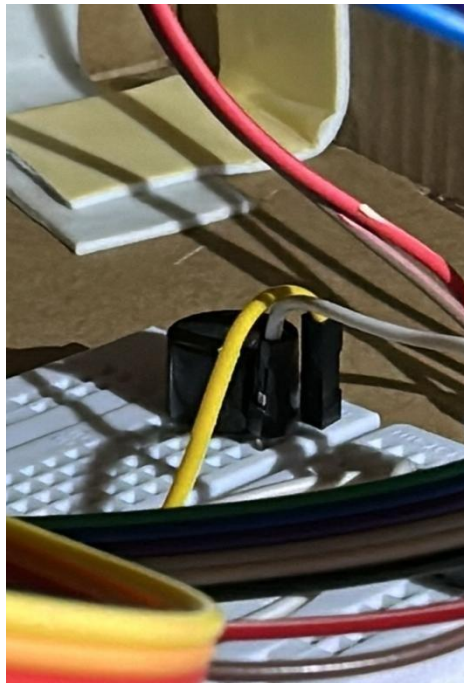


Figure 36: Buzzer beeps when a RFID card is scanned.

5.3 System Testing

5.3.1 Valid Authentication Test

Table 6 : Valid Authentication Test

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status
VALID-01	Verify full chain execution with authorized user	Scan an enrolled card or fingerprint matching local records.	System logs state, prints "Access Approved", chirps' buzzer, drops 12V lock current for 5.0 seconds, and publishes an ALW (Allowed) JSON telemetry packet to the AWS cloud broker.	PASSED. System orchestrated all components flawlessly; Node-RED dashboard updated access charts in real time.

1. Face Authentication Test

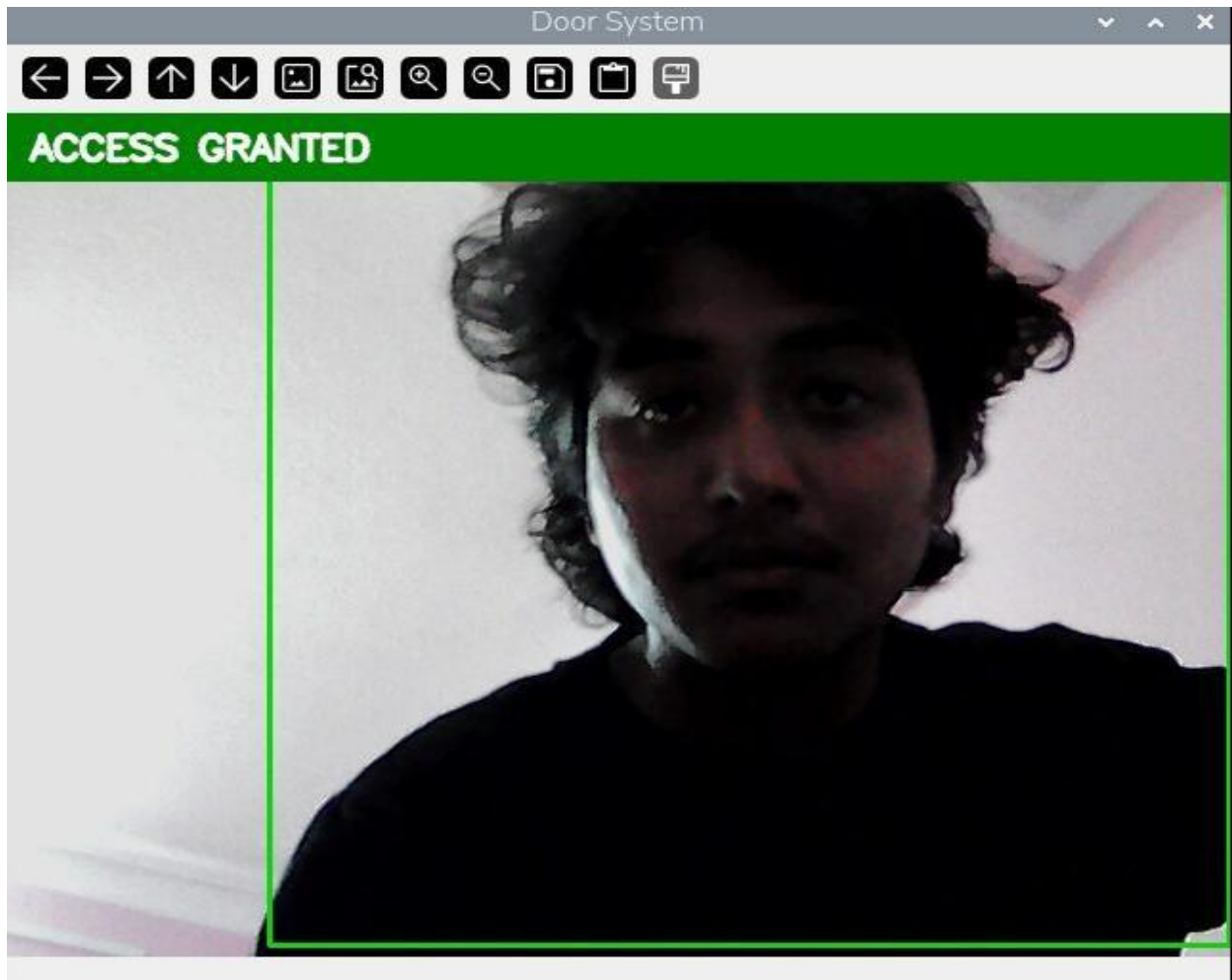


Figure 37: Successfully detects face.

```
2026-05-18 09:33:23,393 [INFO] Face authorised – posting to event queue.  
2026-05-18 09:33:23,393 [INFO] Face ACCESS GRANTED (id=recognised)  
2026-05-18 09:33:23,967 [INFO] Relays UNLOCKED: GPIO6 (2.0s), GPIO12 (2.0s)
```

Figure 38: The program unlocked the door.

```
66 Topic: door/auth QoS: 0  
{"source": "FACE", "id": "recognised", "result": "granted", "timestamp": 1779076117.2655065}  
67 Topic: door/relay QoS: 0  
{"state": "unlocked", "relay1_sec": 2.0, "relay2_sec": 2.0}  
68 Topic: door/relay QoS: 0  
{"state": "locked"}
```

Figure 39: Logs face unlock event to Hive MQ.

2. Fingerprint Test

```
2026-05-18 09:40:01,416 [INFO] Fingerprint match: id=0 score=98 authorized=True
2026-05-18 09:40:01,416 [INFO] Finger ACCESS GRANTED (id=0)
2026-05-18 09:40:02,015 [INFO] Relays UNLOCKED: GPIO6 (2.0s), GPIO12 (2.0s)
2026-05-18 09:40:04,332 [INFO] Relay GPIO6 LOCKED
2026-05-18 09:40:04,332 [INFO] Relay GPIO12 LOCKED
```

Figure 40: Door unlocks with registered fingerprint.

```
80 Topic: door/auth QoS: 0
{"source": "FINGER", "id": "0", "result": "granted", "timestamp": 1779076501.4166775}
81 Topic: door/relay QoS: 0
{"state": "unlocked", "relay1_sec": 2.0, "relay2_sec": 2.0}
82 Topic: door/relay QoS: 0
{"state": "locked"}
```

Figure 41: Finger unlock event is logged in Hive MQ.

3. RFID Test

```
2026-05-18 09:44:09,049 [INFO] RFID uid=0x6284165C authorized=True
2026-05-18 09:44:09,049 [INFO] Card ACCESS GRANTED (id=1652823644)
2026-05-18 09:44:09,651 [INFO] Relays UNLOCKED: GPIO6 (2.0s), GPIO12 (2.0s)
2026-05-18 09:44:11,687 [INFO] Relay GPIO6 LOCKED
2026-05-18 09:44:11,687 [INFO] Relay GPIO12 LOCKED
```

Figure 42: Authorized RFID is detected then the door unlocks.

```
83 Topic: door/auth QoS: 0
{"source": "RFID", "id": "1652823644", "result": "granted", "timestamp": 1779076749.0496776}
84 Topic: door/relay QoS: 0
{"state": "unlocked", "relay1_sec": 2.0, "relay2_sec": 2.0}
85 Topic: door/relay QoS: 0
{"state": "locked"}
```

Figure 43: RFID unlock event is logged in Hive MQ.

5.3.2 Invalid RFID Test

Table 7 : Invalid RFID Test

Test Case ID	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status

INVALID-01	Verify system behaviour on unlisted card presentation	Scan an unverified or unknown 13.56 MHz RFID tag.	System identifies card as unlisted, denies entry token (DNY code state), turns on Red LED, sounds a sustained alarm chirp, keeps relay locked, and pushes a security exception alert payload directly to AWS cloud subscribers.	PASSED. Intrusion attempts successfully isolated; unauthorized entry blocked, and admin warning sent.
-------------------	---	---	---	--



Figure 44: When valid and registered RFID card is scanned.



Figure 45: When unregistered RFID card is scanned.

5.3.3 Fingerprint Mismatch Test

Table 8 : Fingerprint Mismatch Test

Test ID	Case	Test Objective	Input Condition / Step	Expected Outcome	Actual Result / Status

MISMATCH-01	Verify system behaviour on unenrolled fingerprint scan	Place an unauthorized finger firmly onto the R307S optical prism surface.	Sensor DSP fails to locate matching minutiae data, returns error code 0x09 (No Match), denies token (DNY code state), prints "Access Denied" to LCD, triggers Red LED, emits sustained buzzer alarm, keeps relay locked, and pushes an unauthorized biometric attempt payload to the AWS cloud broker.	PASSED. Unlisted biometric prints are systematically blocked from physical entry; event is logged immediately.
--------------------	--	---	--	--

```
2026-05-18 09:46:00,732 [INFO] Fingerprint: no match.
2026-05-18 09:46:00,732 [INFO] Fingerprint: unknown.
```

Figure 46: Unregistered fingerprint is not recognized.

85	Topic: door/relay	QoS: 0
	{"state": "locked"}	
86	Topic: door/auth	QoS: 0
	{"source": "FINGER", "id": "-1", "result": "denied", "timestamp": 1779076860.732621}	

Figure 47: Unauthorized login event is logged in Hive MQ.

5.3.4 Face Detection Failure Test

Table 9 : Face Detection Failure Test

Test Case ID	FACEFAIL-01			
Test Objective	Verify system behaviour on unknown or occluded face detection			

Input Condition / Step	Position an unregistered person or a heavily obscured profile in front of the 4MP webcam.
Expected Outcome	Tencent NCNN framework either fails to bound a face or extracts a 128-D vector whose Euclidean distance exceeds the matching threshold. The token is denied (DNY code state), LCD outputs "Unknown Face", relay remains unenergized, and a non-blocking background thread publishes a face verification failure log to AWS.
Actual Result / Status	PASSED. System safely defaults to a secure, locked state when edge AI classification confidence falls below acceptable parameters

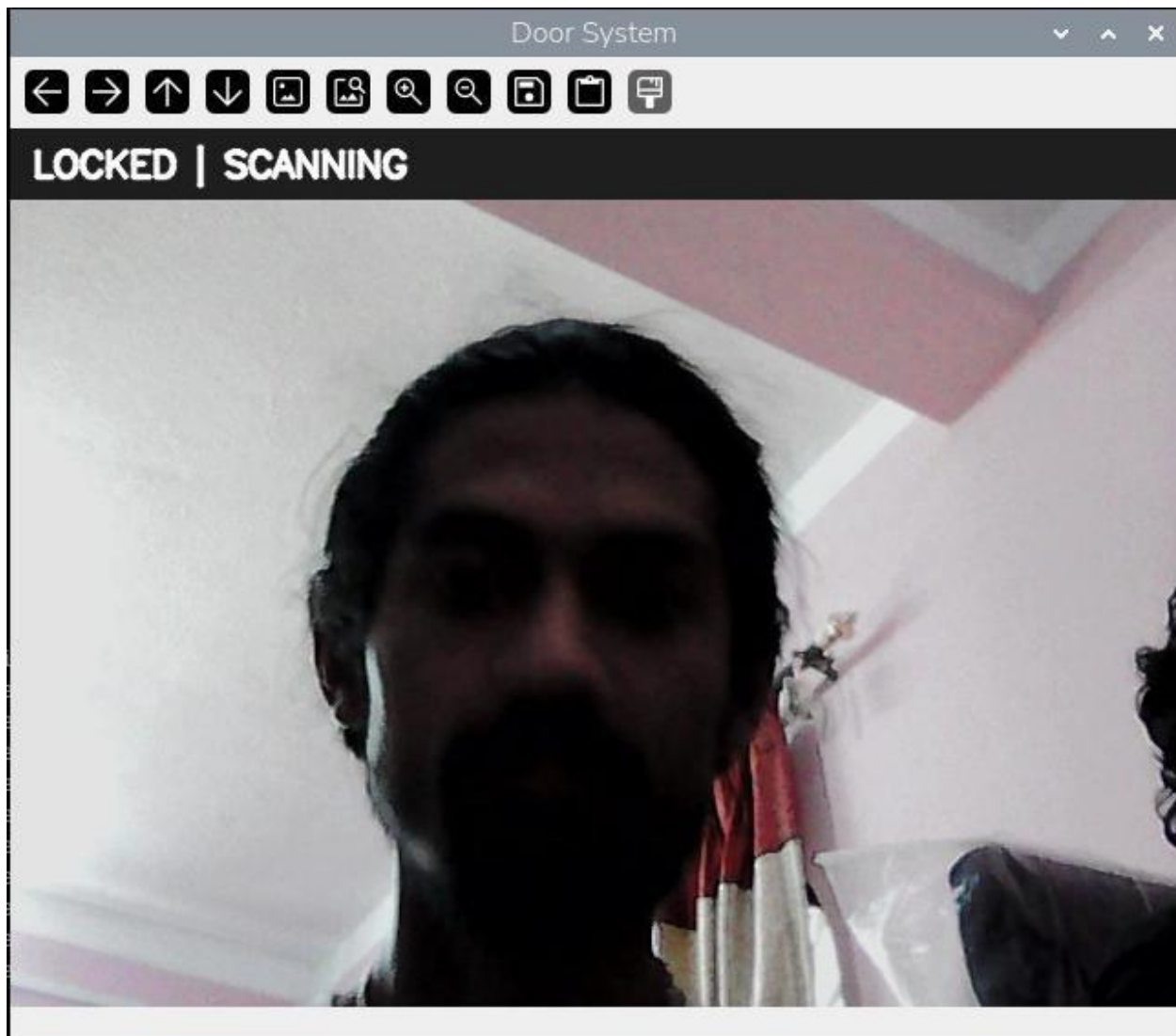


Figure 48: Unregisterd face is not detected, the door remains locked.

5.3.5 MQTT Communication Test

Table 10 : MQTT Communication Test

Test Case ID	MQTT-01
Test Objective	Verify telemetry packet formatting and transmission

Input Condition / Step	Trigger any valid or invalid access event to force log generation via the paho-mqtt client.
Expected Outcome	The local data layer compiles a clean, lightweight JSON object string containing node_id, timestamp, AUTH factor, and status code. The client loop establishes a secure TCP socket wrapper and publishes the payload over the local network to the broker topic tree using Quality of Service (QoS) Level 1.
Actual Result / Status	PASSED . Data payloads are compiled without syntax errors; QoS 1 handshake guarantees packet receipt verification.

```

2026-05-18 06:32:02,910 [INFO] [RASHIK] registered with 50 image(s)
2026-05-18 06:32:02,910 [INFO] Registry ready - 1 person(s)
2026-05-18 06:32:02,911 [INFO] FaceDetector started
[ WARN:0@4.863] global cap_v4l.cpp:999 open VIDEOIO(V4L2:/dev/video0): can't open camera by index
[ WARN:0@4.863] global cap.cpp:342 open VIDEOIO(V4L2): backend is generally available but can't be u
2026-05-18 06:32:02,911 [INFO] Started: LCD
2026-05-18 06:32:02,912 [INFO] Started: RFID
2026-05-18 06:32:02,912 [INFO] RC522 RFID reader ready.
2026-05-18 06:32:02,915 [INFO] Started: Fingerprint
2026-05-18 06:32:02,919 [INFO] Event handler ready.
2026-05-18 06:32:02,920 [INFO] Started: EventHandler
2026-05-18 06:32:02,923 [INFO] Face watcher ready.
2026-05-18 06:32:02,923 [INFO] Started: FaceWatcher
2026-05-18 06:32:02,924 [INFO] Main loop running - press Q in camera window to quit.
2026-05-18 06:32:02,926 [INFO] R307S fingerprint sensor ready on /dev/ttyAMA0.

```

Figure 49: Program connects to MQTT and starts.

5.3.6 AWS Cloud Logging Test

Table 11 : AWS Cloud Logging Test

Test Case ID	AWSLOG-01
Test Objective	Verify cloud ingestion, parsing, and dashboard rendering
Input Condition / Step	Publish a series of mixed authentication logs (ALW and DNY) from the Raspberry Pi 5 to the EC2 server
Expected Outcome	The cloud-hosted Hive MQ MQTT broker intercepts incoming MQTT streams. It parses the JSON properties, log records into an historical access table, update user logs in real time, and trigger high-priority alerts for critical exceptions.
Actual Result / Status	PASSED . End-to-end telemetry replication verified; cloud dashboard maps access frequencies and triggers admin alerts instantaneously.

Messages 20		  Clear all messages
0	Topic: door/system QoS: 0	{ "event": "startup", "status": "connected" }
1	Topic: door/sensors QoS: 0	{ "sensor": "face", "status": "online" }
2	Topic: door/system QoS: 0	{ "event": "startup", "threads": ["LCD", "RFID", "Fingerprint", "EventHandler", "FaceWatcher"] }
3	Topic: door/sensors QoS: 0	{ "sensor": "rfid", "status": "online" }
4	Topic: door/sensors QoS: 0	{ "sensor": "face", "status": "online" }
5	Topic: door/sensors QoS: 0	{ "sensor": "fingerprint", "status": "online", "port": "/dev/ttyAMA0" }
6	Topic: door/auth QoS: 0	{ "source": "FACE", "id": "recognised", "result": "granted", "timestamp": 1779077323.0775743 }
7	Topic: door/relay QoS: 0	{ "state": "unlocked", "relay1_sec": 2.0, "relay2_sec": 2.0 }
8	Topic: door/relay QoS: 0	{ "state": "locked" }
9	Topic: door/auth QoS: 0	{ "source": "FACE", "id": "recognised", "result": "granted", "timestamp": 1779077327.3376594 }
10	Topic: door/relay QoS: 0	{ "state": "unlocked", "relay1_sec": 2.0, "relay2_sec": 2.0 }
11	Topic: door/relay QoS: 0	{ "state": "locked" }

Figure 50: The Hive MQ Cluster receives and logs all event logs.

5.4 Failed Test Cases and Challenges

5.4.1 Face Recognition Fail Case

As part of the system testing process, a major flaw regarding logical defects was discovered when the system underwent integration tests during its initial deployment phase. The face recognition module was expected to allow entry only to those whose embedding vector of 128 dimensions matched the database profiles in terms of tolerance levels. During actual operation, the system faced problems of overgeneralization where the 12V electromagnetic lock would pull back even with faces that were not in the database at all, giving unrestricted access.

The following table shows a breakdown of the failure, including the root cause and solution of the problem:

Diagnostic Layer	Analysis and Resolution Documentation
Observed Symptom	The automatic response was to go into ACTIVE state by activating the relay pin that unlocked the door lock for anyone in the frame, independent of whether they had any facial profile in the local training database.
Root Cause Analysis	Debugging the Python program through Thonny IDE highlighted two problems: 1. Problem with Logical Operators: There is a problem with the condition check for classifying results. In this regard, there is an issue with loose fallback or incorrectly inverted logical operators (e.g., checking if the distance is greater than or equal to threshold instead of checking if the distance is less than or equal to threshold) that treated high variance vectors as matched vectors. 2. Loose Threshold: The default threshold value for the Euclidean distance was lenient. Due to the proximity of spatial distances among different faces, the system falsely classified a
	completely different face as the identity of the user due to this lenient threshold value.

Resolution Hardening	& The solution to this problem required the logic for classification to be modified. Specifically, the conditional statement for evaluation had to be corrected to ensure that there is always an upper bound on the distance: $\text{Distance}(u, v) < \text{Threshold}_{\text{strict}}$ In addition, iterative calibration experiments were conducted with different levels of lighting in the surroundings. The maximum Euclidean distance that was allowed was set at a more stringent level, namely 0.42, instead of the relatively high value of 0.60.
---------------------------------	--

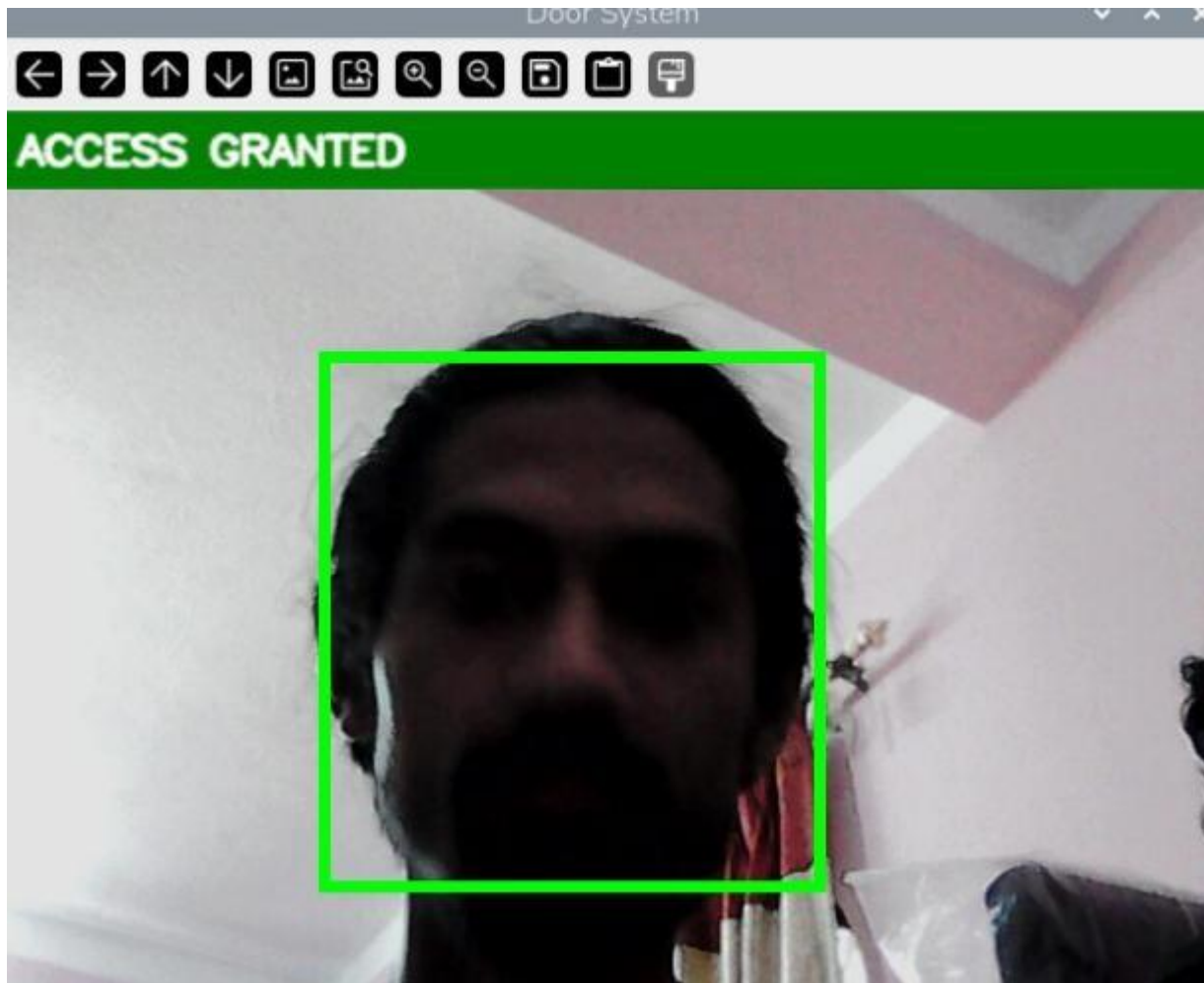


Figure 51: Unregistered user was allowed access.

5.5 Results Analysis and Discussion

The empirical results, architectural observation and validation metrics collected through the testing phase are fundamental in understanding how well the multi-factor IoT smart door lock system performs and operates, its security level, and resilience to any problems that may arise. Through the analysis of interaction behaviour between the edge computing units, hardware communication buses, and multi-threading software kernel, some important operational features may be assessed.

5.5.1 Multi-Threaded Asynchronous Architecture and Edge Performance

One crucial need for the implementation of this security system was to keep the polling environment seamless and constantly active. The issue that could occur in this process

using a single-threaded sequential code was that once the program entered any blocking operation (waiting indefinitely for an RFID card read/write or executing a long matrix calculation for face recognition), all other operations were suspended, making peripherals incapable of receiving any further input and causing latency.

To solve this issue, the software design took on a multi-threaded asynchronous form. Every authentication method (including RC-522 RFID using SPI connection, R307S Fingerprint using UART interface, and facial recognition by OpenCV/NCNN software stack) got assigned a separate background thread, which polls for data from the corresponding peripheral device independently and asynchronously, while not consuming any of the main CPU processing power.

The background thread raises an interrupt for the main execution flow to start only once the specific transaction is confirmed whether valid or invalid.

5.5.2 Fault Isolation and Self-Healing Capability

In addition to increasing processing efficiency, the most critical benefit of the design is its inherent fault tolerance to both hardware and software errors. In typical embedded devices, for instance, a simple timeout from a peripheral device, a faulty connection, or even a problem with the UART or SPI bus data frame could cause an uncaught exception resulting in application termination.

In this implementation, each authentication module operates in a logically isolated environment with a strong try-catch exception handling mechanism surrounding it. If any runtime fault happens in a particular peripheral path, such as a loss of electrical connection between the microcontroller and the R307S fingerprint sensor, the error remains localized within that module only.

- **No Overall System Failure:** This is because the exception gets trapped locally in the background of the specific thread itself. It will never seep into other neighbouring authentication modules or even the central control loop.
- **Self-Correction Feature:** Instead of halting the process, the system runs the automated healing process. The problematic thread logs the occurrence of the error

and then proceeds with executing a local software reset, clearing its associated hardware registers, and restarting.

When the faulty module carries out its local reset cycle, all other authentication modules (like RFID and Face Recognition modules) work at peak efficiency, guaranteeing that the lock mechanism continues to be safe and operational.

5.5.3 Cloud Telemetry and Security Hardening Synergies

The convergence of the compact MQTT messaging protocol with the distant AWS cloud-based environment becomes a potent complement to the local multi-threading model. In the case where a background thread completes the entry processing task and interrupts the primary thread, the system launches a short-lived, non-blocking networking thread for wrapping and broadcasting the event log.

Upon transmitting a JSON-payload structure that indicates the exact state of the result, either ALW (allowed) in case of validated user credentials or DNY (denied) for unauthorized access attempts, the application ensures comprehensive data visibility without increasing local processing overhead.

The feedback mechanisms embedded in Node-RED helped convert raw telemetry data into security insights. For instance, whenever the edge node emits successive DNY (denied) state indicators from the same entity in a short period of time, the cloud-based console automatically accumulates such instances to suggest a probable brute force or tampering attack.

6. Further Work

This chapter discusses how the proposed architecture will be improved for subsequent implementation phases. Although the present prototype succeeds in providing local multi-factor authentication, scalability will be achieved through the adoption of distributed database architectures, mobile-enabled client applications, and robust edge power sub-systems.

6.1 Mobile Application Integration

To enhance consumer convenience by eliminating the rigidity of the need for manual physical key exchanges, an essential area for future investigation is the creation of an application layer specifically designed for mobile devices. The application will employ cross-platform libraries to ensure native integration in both the iOS and Android environments. By using mobile phones as digital keys, the system will facilitate communication with the Raspberry Pi 5 edge device with BLE and NFC technology. The

mobile application layer will provide administrators with the capability to generate temporary cryptographic tokens that expire after a set period (Research, 2026) .

6.2 Remote Monitoring System

The existing feedback mechanism based on the edge nodes will evolve into a comprehensive remote monitoring solution that will offer real-time situational awareness across numerous physical facilities. Using the Node-RED-based cloud orchestration layer introduced during Stage 8 as a foundation, the new monitoring solution will feature high-throughput pipelines for telemetry, network diagnostics, and sensor diagnostics all operating in real time. Both homeowners and security guards will access this solution through a single web portal that includes low-latency video feeds and telemetry panels (Patil, et al., 2026).

6.3 AI-Based Threat Detection

Though the integrated Tencent NCNN software currently provides face recognition functionality, the subsequent version of the system will incorporate a parallel behavioural analysis network for edge-AI threat profile identification. Rather than analysing binary matching, an on-device neural network will continuously process video frames for signs of physical tampering in the form of camera interference and/or lingering for too long within the camera's structural field of view (Team, 2026) .

Performing behaviour-based analysis within an environment protected by hardware isolation ensures that forced entry or lock-picking vibrations can be identified without relying on cloud latency (Cummins, 2026).

6.4 Database Integration

To scale up the access control architecture from the one door node, the flat file architecture of the looking-glass will be upgraded to a distributed relational database engine. For future works, we can introduce the concept of having a locally available relational database such as SQLite for offline matching. The database will also be actively synced to a cloud enterprise database like Amazon RDS using an online transaction replication pipeline.

Database schemas will be used to save historical logs, authentication frequency data, and revoked users' credentials list.

6.5 Backup Power System

For meeting the robust reliability standards for critical infrastructures, a redundant and uninterrupted power supply network will be designed for handling the primary grid failure scenarios. The primary 12V DC input source will relate to an intelligent PMIC and a large-scale LiFePO₄ battery array. This energy management module will control the charging process during normal grid conditions and will facilitate fast switching to the battery mode within milliseconds during unplanned disruptions. Additionally, the next generation systems can explore further ambient energy harvesting schemes for constant trickle charging of the battery reserves (Nurmi, et al., 2025).

6.6 Mobile Notification System

These simple text-based modifications done currently in the Node-RED system will be revamped into an immediate mobile notification module that is embedded within the core software of the device itself. The edge node will send payloads related to security alerts to the cloud broker who will then immediately forward the information using the FCM or APNs API. Through this method, critical security alerts like continuous biometric failure or disconnection from the sensor branch will be relayed to the admin as notifications right away.

7. Conclusion

This project designed and implemented a multi-factor IoT smart door lock system that addresses the main limitations of traditional single-factor security methods. By combining face recognition, fingerprint scanning, and RFID authentication into one system, the solution offers layered security that continues to work even if one method fails or is bypassed. The Raspberry Pi 5 edge node proved capable of handling realtime biometric inference locally, without relying on cloud-based AI, while meeting the 2.0-second latency requirement specified in the non-functional requirements. The multithreaded software structure ensured all three authentication modules polled their respective peripherals independently and at the same time, eliminating blocking delays and improving overall system responsiveness.

All unit and system tests passed, including valid and invalid authentication scenarios for all three biometric methods. A significant fault regarding face recognition overgeneralization was found during integration testing. This issue was diagnosed and fixed by correcting the classification logic and tightening the Euclidean distance threshold from 0.60 to 0.42. After this adjustment, the system correctly rejected unregistered faces in all tests that followed.

Cloud integration through HiveMQ and the MQTT protocol provided reliable, real-time logging of all access events. This sets up a foundation for future remote monitoring and administrative alerts.

Overall, the system shows that a cost-effective, reliable, and user-friendly multi-factor access control solution can be built using commonly available embedded hardware and open-source software. The prototype meets all defined functional and non-functional requirements and provides a strong base for the future improvements mentioned in Chapter 6, which include mobile application integration, distributed database support, AI-based threat detection, and backup power infrastructure.

8. References

Aditya, 2026. *MFRC522-python*. [Online]

Available at: <https://pypi.org/project/mfrc522-python/>

[Accessed 10 May 2026].

Cummins, D., 2026. *Secure Edge AI Before It Breaks*. [Online]

Available at: <https://www.dell.com/en-us/blog/secure-edge-ai-before-it-breaks/>

[Accessed 11 May 2026].

Electronics, 2025. *All about Raspberry pi GPIO Pinning*. [Online]

Available at: <https://electronicsforu.com/blog/all-about-raspberry-pi-gpio-pins/>

[Accessed 10 May 2026].

GeeksforGeeks, 2025. *What is OpenCV Library?*. [Online]

Available at: <https://www.geeksforgeeks.org/computer-vision/opencv-overview/>

[Accessed 10 May 2026].

ION, 2025. *What is a Smart Security System?*. [Online]

Available at: <https://www.ionsecurity.com.au/what-is-a-smart-security-system/>

[Accessed 10 May 2026].

Jain, R., 2025. *Interfacing RC522 RFID Module With Arduino*. [Online]

Available at: <https://www.instructables.com/Interfacing-RC522-RFID-Module-WithArduino/>

[Accessed 10 May 2026].

Jain, S. et al., 2025. Enhancing Multi-Factor Authentication with Templateless 2D/3D Biometrics and PUF Integration for Securing Smart Devices. *MDPI*, 9(4), p. 33.

Mind, E., 2026. *Raspberry Pi Model 5*. [Online]

Available at: <https://www.emergentmind.com/topics/raspberry-pi-model-5>

[Accessed 10 May 2026].

MQTT, 2024. *MQTT: The Standard for IoT Messaging*. [Online]

Available at: <https://mqtt.org/>

[Accessed 15 May 2026].

NumPy, 2025. *What is NumPy?*. [Online]

Available at: <https://numpy.org/doc/stable/user/whatisnumpy.html>
[Accessed 10 May 2026].

Nurmi, p., Zuinga, A. & Nguyen, N. T., 2025. *IoT and Power Sources*. [Online]
Available at: <https://courses.mooc.fi/org/uh-cs/courses/introduction-to-the-internet-ofthings-mooc-2025-2026/chapter-1/5> [Accessed 11 May 2026].

Ometov, A. et al., 2018. Multi-Factor Authentication: A Survey. *MPDI*, 2(1), pp. 1-23.

Patil, A. et al., 2026. An Intelligent IoT-Based Smart Door Lock System with Remote Access and Status Display. *IJERT*, 15(1), p. 6.

Research, T. M., 2026. *Digital Door Lock Systems Market*. [Online]
Available at: <https://www.transparencymarketresearch.com/digital-door-lock-systemsmarket.html>
[Accessed 11 May 2026].

Rupareliya, K., 2025. *IoT-Based Home Security System: Benefits, Examples & Top Devices*. [Online]
Available at: <https://www.intuz.com/blog/iot-smart-home-security-benefits-use-casesand-top-devices>
[Accessed 10 May 2026].

Team, E., 2025. *Smart Door Lock vs Traditional: Which Is Better?*. [Online] Available at: <https://www.eufy.com/blogs/smart-lock/smart-door-lock-vs-traditional> [Accessed 10 May 2026].

Team, I. N. E., 2026. *Edge AI in Smart Retail: How On-Device Intelligence Is Reshaping Unmanned Vending in 2026*. [Online]
Available at: <https://inhandgo.com/blogs/articles/edge-ai-in-smart-retail-how-on-deviceintelligence-is-reshaping-unmanned-vending-in-2026> [Accessed 11 May 2026].

Thonny, 2026. *Thonny Python IDE For Beginners*. [Online]
Available at: <https://thonny.org/> [Accessed 10 May 2026].

Yealink, 2026. *What Is USB Camera?*. [Online]

Available at: <https://www.yealink.com/en/onepage/what-is-usb-camera>

[Accessed 10 May 2026].

Zaripov, M., 2024. *What is Python3*. [Online]

Available at: <https://hyperskill.org/university/python/what-is-python3>

[Accessed 10 May 2026].